Embedded Artificial Intelligence in Battery Management Systems: Pruning and Quantization for Efficient State-of-Charge and State-of-Health Estimation

Florian Rzepka[1,], Julia Kowal[1]

[a]*Technical University of Berlin, Chair of Electrical Energy Storage Technology, Einsteinufer 11, Berlin, 10587, Berlin, Germany*

Abstract

Online battery management requires accurate state-of-charge (SOC) and state-of-health (SOH) estimates within strict memory and timing budgets. This study investigates an engineering application of artificial intelligence (AI): continuous battery-state estimation on an STM32H753ZI microcontroller. The implemented AI consists of stateful long short-term memory (LSTM) networks with multi-layer perceptron (MLP) output heads. Using an open lithium iron phosphate (LFP) ageing dataset, we compare full-precision Base models with structured-pruned and recurrent-weight-quantized variants under identical long-horizon replay and hardware measurements. This paired, auditable compression benchmark is the AI contribution of the work. SOC mean absolute error (MAE) ranges from 2.3 to 2.8 percentage points, while SOH MAE ranges from 0.9 to 1.5 percentage points. Pruning reduces flash by 40.9 % for SOC and 45.5 % for SOH and reduces inference time by 42.9 % and 44.0 %, respectively. Quantization reduces flash by 50.2 % and 58.8 %, but the mixed-precision kernel increases inference time. Estimated energy is a constant-power timing proxy, not a separate power measurement. The quantitative conclusions apply to the evaluated LFP data, checkpoints, firmware, and build settings; other chemistries and controller families require separate validation.

Keywords: Embedded artificial intelligence, Battery state estimation, Long short-term memory network, Structured pruning, Weight quantization, Microcontroller deployment

Nomenclature

Acronyms

BMS	Battery management system
SOC	State of charge
SOH	State of health
DoE	Design of Experiments
LFP	Lithium iron phosphate
LSTM	Long short-term memory network
MLP	Multi-layer perceptron
STM32	32-bit ARM Cortex-M microcontroller family
MAE	Mean absolute error
RMSE	Root-mean-square error
MAC	Multiply–accumulate operation
INT8	Signed 8-bit integer representation (quantized weights)
FP32	32-bit floating-point representation

*Corresponding author

Symbols

t	Time
$U(t)$	Terminal voltage at time t
$I(t)$	Current at time t (positive for discharge)
$T(t)$	Cell temperature at time t
$\mathrm{EFC}(t)$	Equivalent full cycles accumulated up to t
$Q_c(t)$	Cumulative charge throughput up to t
C_{nom}	Nominal discharge capacity of the cell
C_{app}	Apparent capacity inferred from the latest check-up
DoD	Depth of discharge
$\mathrm{SOC}(t)$	State of charge at time t
SOH_k	State of health at reference cycle k
$\mathbf{x}_t$	Input feature vector at time t
$\mathbf{h}_t$	LSTM hidden state at time t
$\mathbf{c}_t$	LSTM cell state at time t
$\mathbf{z}_t$	Concatenation of input and previous hidden state at time t
$\mathbf{i}_t, \mathbf{f}_t, \mathbf{o}_t, \mathbf{g}_t$	Input, forget, output, and candidate gates at time t
$\sigma(\cdot)$	Logistic sigmoid activation function
$\tanh(\cdot)$	Hyperbolic tangent activation function
$\odot$	Element-wise (Hadamard) product
W	Weight matrix (LSTM or MLP)
b	Bias vector
q	Quantized weight (INT8 code)
s	Per-row quantization scale (FP32)
$\hat{w}$	Reconstructed floating-point weight after dequantization
ε_q	Positive lower bound for the quantization scale
$\mathcal{R}$	Index set of retained LSTM hidden units
e_n	Signed prediction error of sample n in percentage points
$Q_p(\cdot)$	Empirical quantile operator at probability p
$W_{\mathrm{ih}}^{\mathrm{int8}}$	Quantized input-to-gate weight matrix
$W_{\mathrm{hh}}^{\mathrm{int8}}$	Quantized hidden-to-gate weight matrix
$s_{\mathrm{ih}}, s_{\mathrm{hh}}$	Per-row quantization scale vectors for $W_{\mathrm{ih}}^{\mathrm{int8}}$ and $W_{\mathrm{hh}}^{\mathrm{int8}}$
b_{fp32}	Floating-point bias of a given gate
g	Gate pre-activation used in the quantized LSTM kernel
D, H, M	Input dimension, LSTM hidden size, and MLP width
N_{MAC}	Analytical multiply–accumulate count per inference

1. Introduction

Accurate online estimation of the state of charge (SOC) and the state of health (SOH) remains a central challenge for battery-powered systems [**? ? ? ?**]. Recent reviews show sustained progress in model-based and data-driven estimators, but they also emphasise that accuracy depends strongly on cell chemistry, operating conditions, reference construction, and evaluation protocol [**? ?**]. For an embedded battery management system, predictive accuracy alone is therefore insufficient: the estimator must also fit the available flash and RAM, meet the update deadline, and preserve its state consistently over long input streams.

The study is motivated by modular stationary storage systems in remote microgrids. Within the MG Farm programme [**? ? ? ?**], partners from France, Algeria, Morocco, and Germany develop smart-grid demonstrators for North African sites. The Technical University of Berlin contributes the battery ageing experiments, the LFP DoE Cycle Ageing Dataset, a battery management system prototype, and the SOC/SOH estimators deployed on its controller. Each storage module must operate autonomously within the flash, RAM, and timing limits of an STM32H753ZI-class microcontroller. Similar constraints arise in mobile systems that cannot rely on continuous cloud access.

Existing embedded studies demonstrate that compact neural SOC or SOH estimators are feasible on microcontrollers and that quantization can reduce their storage requirements [**? ? ? ? ?**]. However, these studies generally examine one battery state, one model family, or one deployment platform. They do not provide a common, auditable comparison of structured hidden-unit pruning and recurrent-weight quantization for both stateful SOC and SOH estimators under identical long-horizon replay and hardware measurement conditions. Reproduction is further hindered when scaling parameters, recurrent-state handling, and the exact mixed-precision boundary are not reported.

This work uses the Lithium Iron Phosphate (LFP) DoE Cycle Ageing Dataset [**?**]. It contains long-term cyclic ageing measurements from multiple LFP cells under full and fractional factorial conditions, with controlled temperature, charge and discharge rates, depth-of-discharge variations, and interleaved capacity, open-circuit-voltage, and pulse-power check-ups. These data support a paired assessment of rapid SOC dynamics and slowly evolving SOH trajectories.

The objective of this study is to quantify, under identical data and deployment conditions, how structured hidden-unit pruning and recurrent-weight post-training quantization change the accuracy, memory footprint, and inference cost of stateful LSTM + MLP SOC and SOH estimators on an STM32H753ZI. The study makes three contributions:

(i) an auditable path from trained PyTorch checkpoints to stateful C inference, including explicit feature scaling, recurrent-state handling, pruning, and mixed-precision quantization;

(ii) a controlled Base–Pruned–Quantized comparison for both SOC and SOH using continuous replay, linked-firmware flash and RAM, host latency, and cycle-counter-measured kernel time; and

(iii) an evidence-bounded trade-off analysis that connects architecture-level operation counts and implementation details to long-horizon accuracy, filtering, and application-dependent utility.

The novelty lies in this paired, transparent deployment and compression benchmark rather than in proposing a new recurrent cell or compression algorithm.

2. Related Work and Motivation

2.1. SOC/SOH Estimation

Existing SOC/SOH estimation methods can be grouped into three families [**? ? ? ?**]. First, *measurement-derived* techniques use Coulomb counting and open-circuit-voltage (OCV) look-up tables during rest periods. They provide inexpensive proxies but suffer from drift, offset accumulation, and a strong dependence on initialisation and periodic recalibration. Second, *model-based* observers combine equivalent-circuit or electrochemical models with Kalman filtering and its variants. These approaches can offer physical interpretability but require careful parameter identification and are sensitive to model mismatch and ageing-induced parameter drift. Third, *data-driven* methods employ machine-learning regressors, including LSTM networks, Transformers, and hybrid structures, to map sequences of current, voltage, and temperature to SOC and SOH estimates. Key challenges for such models are robustness when operated under conditions that differ from the training data and the careful treatment of normalisation and state handling on embedded targets.

Recent hybrid estimators combine physical or statistical preprocessing with learned correction models. Wang et al. [**?**] couple online parameter identification with an adaptive radial-basis correction and differential support-vector

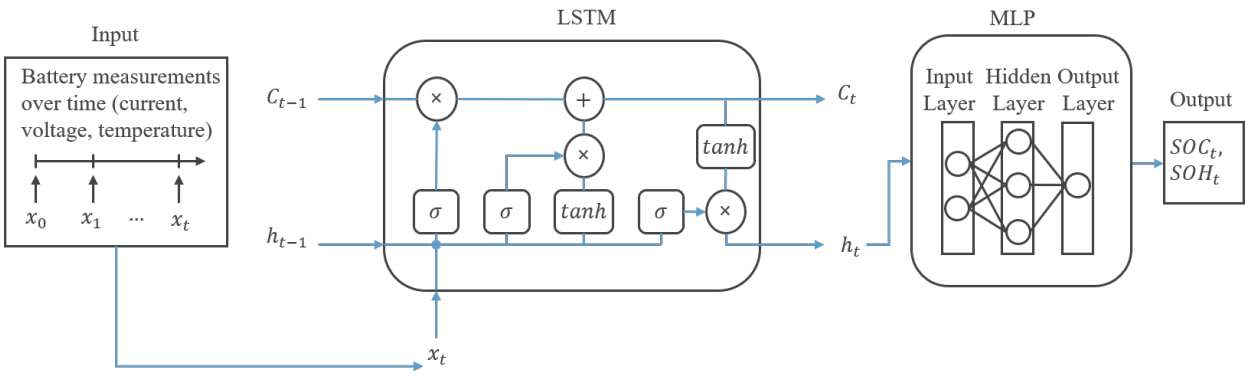


Figure 1: Schematic of the deployed SOC/SOH estimator. Preprocessed feature streams are fed into a single-layer LSTM whose hidden state is propagated across time; at each step a lightweight MLP head maps the hidden representation either to SOC or SOH, enabling stateful streaming inference on the STM32.

machine for SOC estimation under dynamic vehicle-oriented tests. For whole-life capacity estimation, Wang et al. [**?**] combine singular filtering, Gaussian-process regression, and an LSTM to address fast ageing and varying currents. These studies broaden the operating conditions and model structures considered for battery-state estimation. Their primary evidence concerns estimation accuracy, however; neither reports a paired post-training pruning and quantization study with linked-firmware memory and recurrent microcontroller timing. They are therefore complementary to, rather than substitutes for, the deployment benchmark developed here. Across these families the reported accuracy spans a wide range. Electrochemical or Equivalent Circuit Model (ECM)-based observers with carefully tuned filters often achieve SOC root-mean-square errors (RMSE) well below 1 % of full scale and sub-percent SOH capacity errors on laboratory test cycles [**? ? ? ?**]. Recent sequence-model approaches based on N-BEATS and Bidirectional LSTMs (BiLSTMs) for sodium-ion cells report SOC Mean Absolute Error (MAE) between roughly 0.1 and 0.4 % [**? ?**], whereas TinyML-style SOC estimators deployed on microcontrollers often operate in the 1–3 % error regime [**? ? ?**]. These values provide context rather than a direct ranking because chemistry, data splits, target construction, drive cycles, and error definitions differ between studies. Kalman filters, particle filters, and equivalent-circuit-model observers therefore dominate the classical literature, yet their accuracy can degrade over long operating periods with pronounced current and temperature gradients [**? ?**]. Recurrent neural networks, and in particular LSTM-based regressors, address these limitations by integrating long temporal context and absorbing residual modelling errors [**? ?**]. In this work we deliberately restrict the architecture search to a single-layer LSTM with a fully connected Multi-Layer Perceptron (MLP) output layer. LSTMs retain explicit cell states with separate input, forget, and output gates, which made it straightforward to hand-code a numerically stable, stateful kernel in C and to reuse the same backbone for both SOC and SOH. Gated recurrent units (GRUs) or Transformer variants could offer alternative trade-offs between accuracy and cost, but exploring multiple architectures would dilute the focus on the embedded tool flow and is left as future work. Most publications on LSTM-based estimators maximise accuracy with complex architectures and rarely examine how the resulting models behave once deployed on constrained controllers. Even within the LSTM family, sliding-window inference is commonly employed because it delivers high accuracy, but it also requires substantial RAM to buffer multiple past samples. A stateful LSTM that hands over its hidden and cell vectors step by step retains the temporal information without maintaining large windows and therefore forms the first optimisation step in our architecture. The associated handling of hidden states, feature scaling, and numerical stability is rarely described in the literature, which is why this work explicitly targets embedded SOC/SOH deployment.

2.2. Embedded Deployment Strategies

Commercial toolchains such as ONNX Runtime, TensorRT, and STM32Cube.AI support many neural-network deployment workflows [**? ? ?**]. For continuously stateful estimators, however, a generated binary does not always expose recurrent-state handling or the precise placement of mixed-precision scaling operations needed for an implementation audit. Naguib et al. [**?**] compare accuracy, execution time, flash, and RAM for classical and neural SOC

estimators on two NXP microcontrollers. Crocioni et al. [**?**] deploy compact recurrent capacity estimators on STM32 targets and study 8-bit quantization. More recent TinyML studies evaluate quantized or hardware-accelerated SOC estimators on embedded platforms [**?** **?**], while Kamali et al. [**?**] target computationally compact SOH estimation. These studies establish embedded feasibility, but each addresses a narrower combination of task, architecture, compression method, and target hardware.

At the compression-method level, PQ Bench [**?**] systematically compares pruning and quantization across general deep-learning workloads and confirms that their accuracy–storage trade-offs are model dependent. It does not address the continuous recurrent state, SOH post-processing, or microcontroller timing behaviour of battery estimators. The resulting research gap is therefore not the absence of compact neural estimators, but the absence of a controlled, task-paired benchmark that traces two transparent compression routes from saved SOC and SOH checkpoints to measured stateful firmware behaviour. Our study addresses this gap with the same input dimension, streaming protocol, performance-indicator definitions, and STM32 measurement procedure for Base, Pruned, and Quantized variants of both tasks.

To retain control over the implementation boundary, we extract the PyTorch tensors, remove complete recurrent channels according to the documented saliency score, apply per-row symmetric quantization to the recurrent weights, and implement the LSTM gates and activations in C. This path makes the state update, scaling operations, and stored parameter types inspectable and links the model transformations directly to the reported flash, RAM, and timing measurements.

3. Dataset and Preprocessing

3.1. Ageing Experiments and Load Profiles

All experiments rely on the high-resolution LFP DoE Cycle Ageing Dataset [**?**]. The measurements originate from a design-of-experiments study on commercially available JGCFR18650-1800mAh-3.2V LFP cells [**?**] conducted at the Technical University of Berlin. Cells were cycled inside a climatic chamber at 25°C while the following factors were systematically varied:

$$C_{\mathrm{chg}} \in [0.5\,\mathrm{C},\ 0.9\,\mathrm{C}], \tag{1}$$

$$C_{\mathrm{dis}} \in [1.0\,\mathrm{C},\ 2.5\,\mathrm{C}], \tag{2}$$

$$\mathrm{DoD} \in [45\%,\ 65\%]. \tag{3}$$

The mixed full/fractional factorial plan distributes fifteen test cells across eight operating-point combinations and includes replication to assess repeatability. Figure **??** shows the low and high levels of all three factors.

Between ageing loops, dedicated check-ups provide capacity tests, open-circuit-voltage sweeps, and hybrid pulse-power characterisation. Stationary-storage profiles are additionally injected to emulate realistic power flows [**?**]. These profiles originate from residential photovoltaic home-storage applications and contain stochastic load and generation patterns that excite the cell differently from constant-current cycling.

The raw dataset contains uniformly sampled measurements at 1 Hz, including voltage, current, terminal temperature, and auxiliary channels. Partitioned Parquet tables support efficient retrieval of the long ageing sequences. The dataset is published in [**?**]; the cube notation follows standard DoE conventions as summarised in [**?**]. Together, Figures **??** and **??** illustrate the diversity of SOC transients and ageing behaviour that the estimators must capture: short-term SOC dynamics are driven by fast current swings and mode changes between profiles, whereas SOH evolves slowly over hundreds of days and depends on the cumulative operating conditions of each cell.

3.2. Feature Engineering

The SOC model consumes the six-dimensional feature vector

$$\mathbf{x}_t^{\mathrm{SOC}} = \left[U(t),\ I(t),\ T(t),\ Q_c(t),\ \tfrac{\mathrm{d}U}{\mathrm{d}t}(t),\ \tfrac{\mathrm{d}I}{\mathrm{d}t}(t)\right], \tag{4}$$

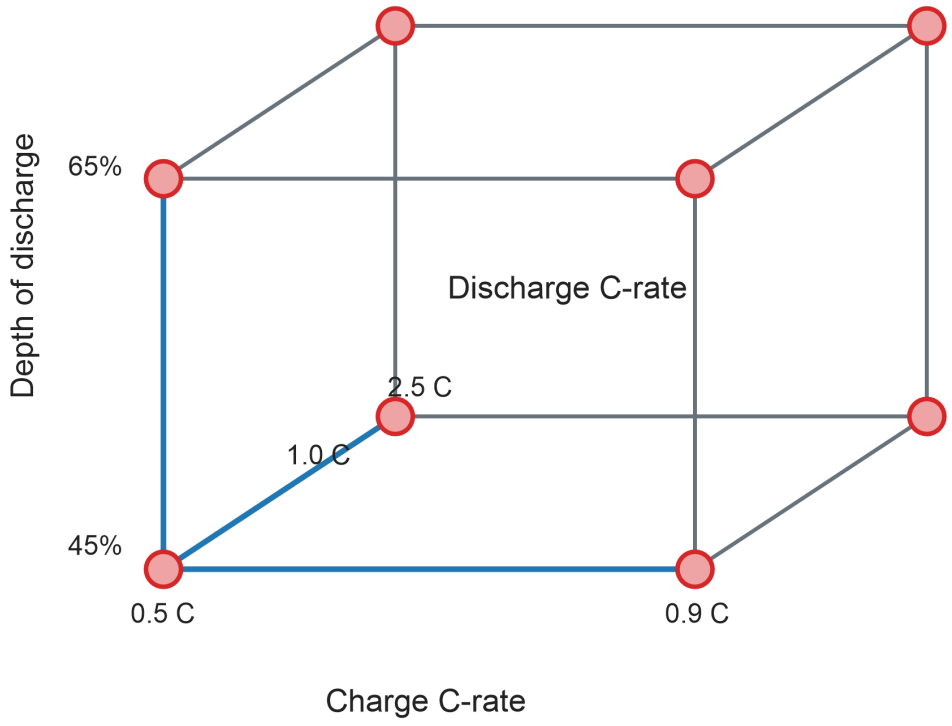


Figure 2: Three-factor design space of the LFP DoE Cycle Ageing Dataset. The cube axes show the low and high levels of charge C-rate, discharge C-rate, and depth of discharge; the eight marked corners are the operating-point combinations across which the fifteen test cells are distributed.

where the audited preprocessing uses timestamp-aware backward differences rather than centred differences. For sample index $k > 0$,

$$\Delta t_k = \max(t_k - t_{k-1}, \varepsilon_t), \qquad \varepsilon_t = 10^{-6}, \tag{5}$$

$$\left.\frac{\mathrm{d}U}{\mathrm{d}t}\right|_k = \frac{U_k - U_{k-1}}{\Delta t_k}, \qquad \left.\frac{\mathrm{d}I}{\mathrm{d}t}\right|_k = \frac{I_k - I_{k-1}}{\Delta t_k}. \tag{6}$$

This causal calculation requires only the preceding voltage/current sample and timestamp and has constant work per update. In the evaluated benchmark, however, derivative and cumulative channels were prepared host-side before replay. Complete six-feature vectors were transmitted via UART, and the firmware applied the same robust-scaling parameters used during model development. Consequently, the reported DWT kernel times exclude sensor acquisition and feature construction; the exact placement of the six-feature scaling operation is documented with the STM32 measurement interval below. Each component is normalised using a robust scaling operation based on the median m and interquartile range r:

$$\tilde{x}^{\mathrm{SOC}}_{t,k} = \frac{x^{\mathrm{SOC}}_{t,k} - m_k}{r_k + \varepsilon}, \qquad \varepsilon = 10^{-6}. \tag{7}$$

The same (m_k, r_k) pairs are embedded in the firmware so that host and STM32 inputs are normalised identically. The SOH model uses a slightly different feature stack to capture slow-varying degradation markers:

$$\mathbf{x}^{\mathrm{SOH}}_t = [t,\ U(t),\ I(t),\ T(t),\ \mathrm{EFC}(t),\ Q_c(t)], \tag{8}$$

where t is the cumulative time stamp and $\mathrm{EFC}(t)$ denotes the equivalent full cycles accrued up to sample t. These channels are likewise robustly normalised, relying on median and interquartile range rather than mean and variance

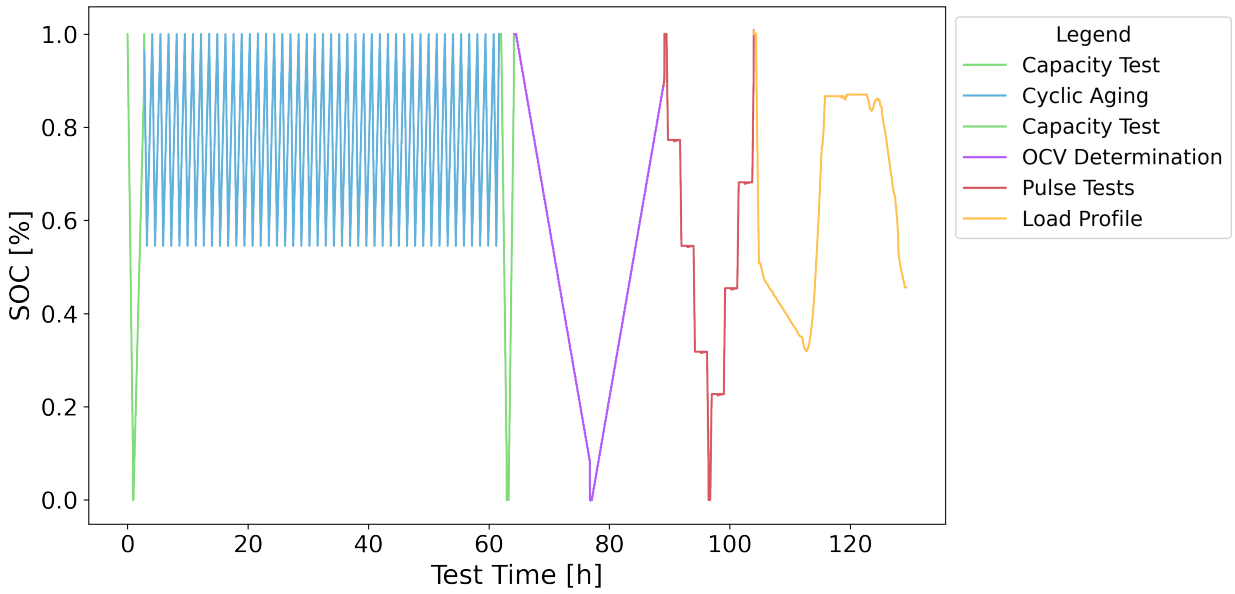


Figure 3: Representative SOC trajectory illustrating selected segments of the experimental protocol used in this study. The plot shows a portion of the cyclic ageing procedure with defined charge and discharge currents and a specified depth-of-discharge (DoD), followed by the diagnostic checkup sequence comprising OCV determination, HPPC pulses, and capacity tests. In addition, the trajectory includes a load profile taken from a stationary battery storage system. Together, these segments capture both controlled laboratory diagnostics and application-oriented operating conditions.

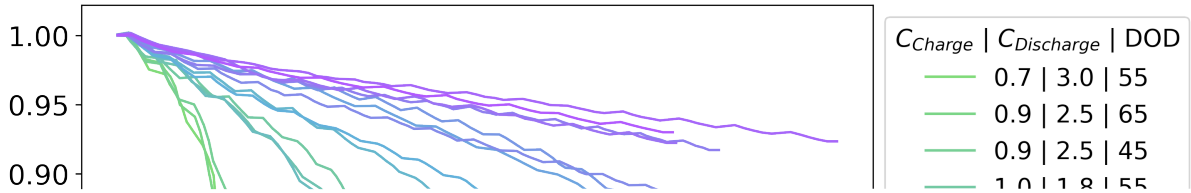


Figure 4: Evolution of the state of health across all LFP cells in the ageing campaign, shown over the total experimental duration. The spread between trajectories reflects the different operating points in the factorial plan (C-rates, depth of discharge, profile mix) and motivates the use of a data-driven SOH estimator that generalises across cells and loading conditions.

to reduce the influence of outliers in current and voltage measurements [**?** **?**]. During deployment, the SOC and SOH models run independently on the microcontroller and each produces one estimate per second. In principle, both estimates could be fused more tightly, but this work keeps the models separate to evaluate their resource use and accuracy independently.

3.3. Reference SOC and SOH targets

The ground-truth SOC trajectory is derived from Coulomb counting with drift compensation. Denoting the apparent cell capacity after a given check-up by C_{app}, the SOC inside a charge-discharge segment is computed as

$$\mathrm{SOC}(t) = \mathrm{SOC}(t_0) - \frac{1}{C_{\mathrm{app}}} \int_{t_0}^{t} I(\tau)\,\mathrm{d}\tau, \tag{9}$$

where the integral is reset to SOC = 1 after each constant-voltage (CV) termination. The reset mitigates the bias that would otherwise accumulate due to capacity fade, yet the long-term traces still reveal gradual changes in the SOC envelope as C_{app} decreases. State of health is obtained from the periodic capacity check-ups through

$$\mathrm{SOH}_k = \frac{C_k}{C_{\mathrm{nom}}}, \tag{10}$$

with C_k representing the measured discharge capacity during the k-th reference cycle (comprising capacity test, open-circuit-voltage sweep, and hybrid pulse-power characterisation) relative to the nominal capacity C_{nom}. These targets highlight the need for a data-driven estimator that remains accurate despite the evolving capacity landscape, which motivates the use of recurrent neural networks for both SOC and SOH predictions. The reference SOC and SOH traces are therefore offline constructs. They rely on information that is not available in deployment, such as long rest periods for OCV stabilisation and capacity from periodic full cycles, and they assume perfect initialisation and recalibration. These constructions follow common practice in battery management, where SOC is obtained by Coulomb counting and SOH is expressed as the ratio between present and nominal capacity [**?** **?** **?**]. On the STM32 the estimators operate purely on the streamed measurements and their internal states; the reference trajectories only serve as ground truth for training and evaluation.

3.4. Streaming inference setup

Because the SOC and SOH estimators are stateful, the LSTM hidden and cell states are continuously forwarded from one sample to the next along the full input stream. At each sampling instant the current feature vector is processed together with the previous hidden and cell states, which yields the new states and the corresponding SOC or SOH estimate. We therefore operate the models in a purely step-by-step streaming regime without forming sliding input windows. On the host the full normalised sequence from the LFP DoE Cycle Ageing Dataset is processed once in this fashion, and the resulting SOC and SOH estimates are aligned with the reference trajectories. On the STM32 firmware the same step-wise LSTM implementation is used, driven by the UART feature stream, so that host and embedded benchmarks reflect identical inference conditions. The streaming dashboards in Figs. **??** and **??** illustrate a complete run for one representative cell.

3.5. SOH prediction post-processing

The SOH trajectories in the final dashboard use two sequential causal filtering stages. First, each raw model output $\hat{y}_t$ is multiplicatively aligned with the initial SOH label, $\hat{y}_t^{(0)} = (y_0/\hat{y}_0)\hat{y}_t$. Stage 1 limits the change of this calibrated prediction relative to the preceding stage output to

$$\ell_t = \min\!\left(10^{-4}|y_{t-1}^{(1)}|,\, 10^{-5}\right), \tag{11}$$

and subsequently applies an exponential moving average (EMA) with $\alpha_1 = 0.02$. The limiter is symmetric, so positive and negative proposal changes are both restricted to $\pm\ell_t$. Stage 2 consumes the complete stage-1 trajectory, applies another EMA with $\alpha_2 = 10^{-6}$, and limits only downward changes according to

$$y_t^{(2)} = \max\!\left(e_t^{(2)}, y_{t-1}^{(2)} - 2\times 10^{-8}\right), \qquad e_t^{(2)} = (1-\alpha_2)e_{t-1}^{(2)} + \alpha_2 y_t^{(1)}. \tag{12}$$

Thus, the curve in Fig. **??** is not obtained by applying α_2 directly to an unfiltered network output.

For an EMA sampled at interval Δt, the equivalent time constant and cutoff frequency are

$$\tau = -\frac{\Delta t}{\ln(1-\alpha)}, \qquad f_c = \frac{1}{2\pi\tau}. \tag{13}$$

At the dataset rate of 1 Hz, stage 1 has $\tau = 49.5\,\mathrm{s}$, $f_c = 3.22\,\mathrm{mHz}$, and an EMA-only 90 % step-response time of approximately 114 s. For stage 2, τ is approximately 11.57 days, $f_c = 1.59 \times 10^{-7}$ Hz, and the corresponding 90 % time is 26.65 days. These frequencies characterise only the linear EMA components; the nonlinear rate limiters have no single cutoff frequency. The very strong second stage can reduce error against slowly varying labels while introducing substantial response delay, which is evaluated separately in **??**.

4. Model Architecture and Baseline Performance

4.1. LSTM architecture in PyTorch

Both SOC and SOH estimators employ the same recurrent equation set but differ in size according to their training releases. The SOC configuration uses $H = 64$ hidden units and an MLP output layer of width 64, whereas the SOH configuration increases both dimensions to 128 to cope with the longer temporal context. LSTM gates i_t, f_t, o_t and candidate state g_t follow the standard formulation used in modern RNN surveys [**?**]. All models are implemented in PyTorch [**?**]. For one streaming update with input dimension $D = 6$, LSTM hidden size H, and fixed MLP width M, the matrix products require

$$N_{\mathrm{MAC}}(H) = 4H(D + H) + HM + M, \tag{14}$$

where one multiply–accumulate (MAC) is counted per weight and bias additions, nonlinearities, indexing, and memory transfers are excluded. This gives 22,080 MACs for SOC Base ($H = M = 64$) and 12,124 after pruning to $H' = 45$, a 45.1 % reduction. The corresponding SOH counts are 85,120 for Base ($H = M = 128$) and 46,208 after pruning to $H' = 90$, a 45.7 % reduction. More generally, a nominal 30 % hidden-size reduction sets $H' = 0.70H$. As the quadratic recurrent term dominates for large H, the analytical MAC reduction approaches $1 - 0.70^2 = 51$ %; the linear input/MLP and fixed output terms keep the implemented models below this limit. **??** visualises this architecture-level scaling. It is not a prediction of identical flash, latency, or energy savings on another microcontroller.

$$\begin{aligned}
&\mathbf{z}_t = [\mathbf{x}_t, \mathbf{h}_{t-1}], \\
&\mathbf{i}_t = \sigma(W_i\mathbf{z}_t + \mathbf{b}_i), \quad \mathbf{f}_t = \sigma(W_f\mathbf{z}_t + \mathbf{b}_f), \\
&\mathbf{o}_t = \sigma(W_o\mathbf{z}_t + \mathbf{b}_o), \quad \mathbf{g}_t = \tanh(W_g\mathbf{z}_t + \mathbf{b}_g), \\
&\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \mathbf{g}_t, \quad \mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t).
\end{aligned} \tag{15}$$

Equations (**??**) and the recurrent update above define the common Base architecture used for both tasks. Pruning changes its hidden dimension and parameter submatrices, whereas quantization changes the stored representation of selected recurrent weights. Neither transformation changes the input features, targets, or streaming state-update order.

Both SOC and SOH models are trained with a mean-squared-error loss between the prediction and the corresponding reference at each training sample. During validation we additionally report mean absolute error (MAE), root-mean-square error (RMSE), and R^2. The trained parameters, together with the corresponding configuration files and optimiser statistics, are archived to enable deterministic replays of both models.

4.2. Training Notes

Training is carried out on a GPU node equipped with NVIDIA A100-class accelerators using mixed-precision (FP16) arithmetic. Both models use the AdamW optimiser with cosine-annealing warm-restart scheduling, gradient clipping, and mini-batch training [**? ? ? ?**]. The SOC configuration uses a learning rate of 1.5×10^{-4}, a batch size of 512, and sequence chunks of length 2024. The corresponding SOH values are 2.0×10^{-4}, 128, and 2048.

Deployment operates the LSTM in stateful, step-by-step streaming mode, whereas training uses truncated back-propagation through time over these chunks. Hidden and cell states are propagated within every chunk in the same temporal order as during deployment. Chunking permits randomised segment sampling and tractable gradient computation without changing the recurrent update equations.

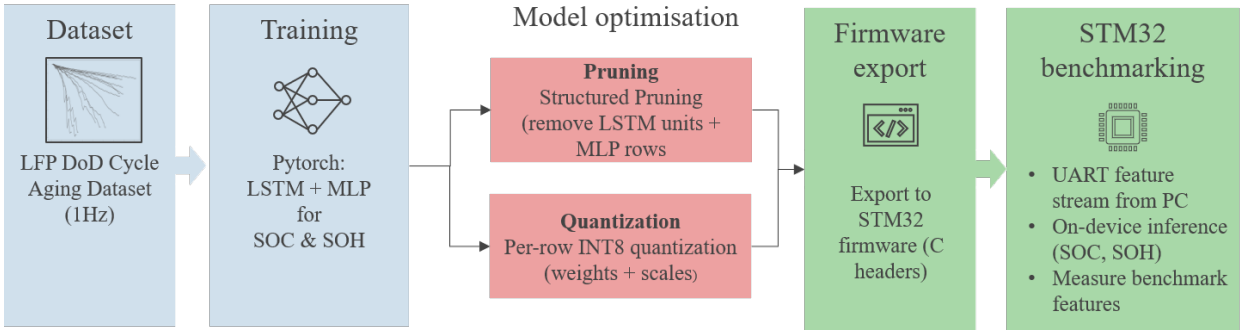


Figure 5: End-to-end pipeline from dataset to embedded benchmarking. The LFP DoE Cycle Ageing Dataset feeds PyTorch training on a GPU workstation; the resulting SOC and SOH LSTM + MLP models are pruned and quantized, exported as C headers, and deployed on an STM32H753ZI board, where flash/RAM usage, inference time, and latency are measured under streaming replay. Estimated energy per inference is derived from kernel time under a constant board-power assumption.

4.3. Baseline Accuracy

On the workstation the SOC and SOH base models are first evaluated in streaming mode on the full ageing trajectory of one representative LFP cell from the LFP DoE Cycle Ageing Dataset [**?**], yielding baseline MAE and RMSE values for both tasks. These Python replays operate on the original PyTorch implementations and serve as a numerical reference for all subsequent experiments. The same checkpoints are then pruned, quantized, and exported to STM32 firmware, where identical feature streams are replayed to measure accuracy, flash and RAM usage, system latency, and on-device inference time. A timing-derived energy proxy is calculated as described in Section **??**. Figure **??** summarises this training-to-firmware benchmarking pipeline.

5. Pruning

Pruning reduces the size and computational cost of a neural network by eliminating parameters or whole units that contribute little to the overall mapping, while keeping the input-output behaviour as close as possible to the dense baseline [**?** **?**]. In our setting we apply structured pruning at the level of complete LSTM hidden units so that the resulting model remains well suited for efficient C implementations on the microcontroller.

5.1. Structured magnitude pruning

Structured magnitude pruning removes the lowest-saliency LSTM units until only 45 hidden channels remain in the SOC model (30% sparsity), with analogous settings for the SOH experiments. Reducing the state dimension is a prerequisite for fitting the estimators into the flash and SRAM budgets of the target STM32-based BMS and for shortening the on-device inference time, inspired by structured sparsity approaches for deep and recurrent networks [**?** **?** **?**]. In analogy to the group-lasso style regularisers proposed for LSTMs in [**?** **?**], which aggregate weights across gates via L2 norms, we define a saliency score for each hidden unit h that combines all incoming weights from the different gates:

$$s_h = \sum_{g\in\{i,f,o,g\}} \left(\|W_{\text{ih}}^{(g)}[h,:]\|_2 + \|W_{\text{hh}}^{(g)}[h,:]\|_2 \right). \tag{16}$$

Units are ranked by s_h, and the H' highest-scoring indices form the retained set $\mathcal{R}$. For each gate g, structured pruning constructs the reduced matrices

$$\begin{aligned} \widetilde{W}_{\text{ih}}^{(g)} &= W_{\text{ih}}^{(g)}[\mathcal{R},:], \\ \widetilde{W}_{\text{hh}}^{(g)} &= W_{\text{hh}}^{(g)}[\mathcal{R},\mathcal{R}], \\ \widetilde{W}_{\text{MLP},1} &= W_{\text{MLP},1}[:,\mathcal{R}]. \end{aligned} \tag{17}$$

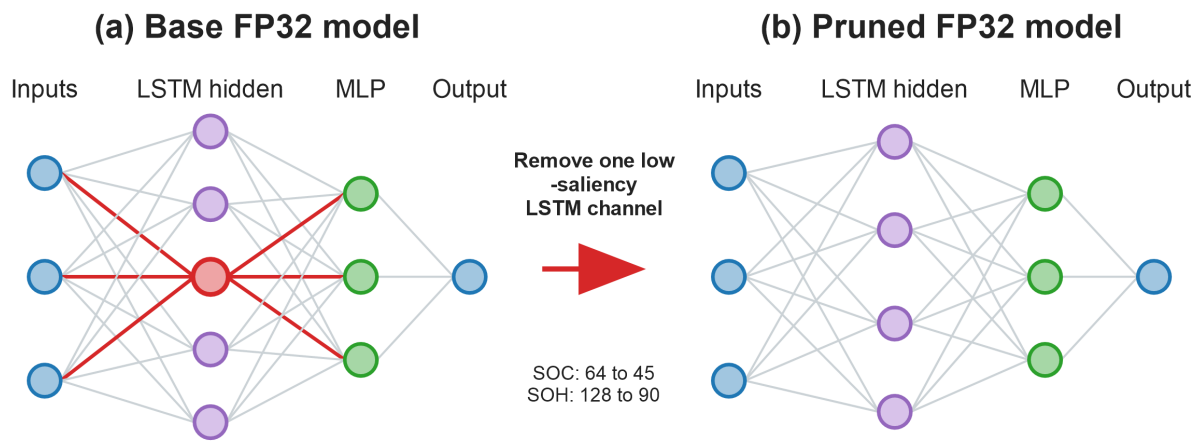


Figure 6: Structured hidden-channel pruning used in this study. Panel (a) marks one low-saliency LSTM channel and its incident connections in red. Panel (b) shows the resulting dense network after removing the corresponding gate rows, recurrent columns, and MLP input column according to Eq. (**??**). The final SOC and SOH hidden dimensions are shown between the panels.

Thus the same index set removes gate rows, recurrent columns, and the corresponding inputs of the first MLP layer. The reduced tensors remain dense; the computational saving comes from the smaller dimension rather than sparse indexing.

The final SOC deployment uses $H' = 45$ instead of 64 hidden units, while SOH uses $H' = 90$ instead of 128. Both transformations remove approximately 30 % of the recurrent channels. This is a one-shot structured operation followed by brief post-pruning fine-tuning; no iterative pruning schedule or pruning-aware training is used. The L2 aggregation gives a gate-consistent ranking that can be reconstructed from the saved weights and mapped directly to dense C arrays. **??** reports the resulting SOC saliency ranking and recurrent-weight distributions. These diagnostics describe the selected checkpoint but do not establish a causal regularisation effect. The criterion was selected because it can be reconstructed from saved weights, combines all four gate rows for one recurrent unit, and maps directly to removal of dense rows, recurrent columns, and the associated MLP input. Gradient sensitivity would additionally require training data and backpropagation, while activation-based ranking would require a representative calibration stream and an explicit aggregation of unit activations. Both alternatives can be formulated as structured criteria, but they were not experimentally compared here. The rationale therefore concerns reproducibility and compatibility with dense dimension reduction, not universal superiority of L2 ranking; **??** summarises this boundary.

5.2. Resulting pruned models

The resulting pruned SOC and SOH models still run in FP32 but reduce flash consumption and runtime considerably once compiled for STM32. Section **??** summarises the corresponding latency and memory savings. These pruned variants are evaluated alongside their base and quantized counterparts throughout the results section.

6. Quantization

Quantization reduces the numerical precision of selected weights while keeping the network topology unchanged. We apply post-training, weight-only quantization to the LSTM input-to-hidden and hidden-to-hidden matrices. Biases, activations, recurrent hidden and cell states, and the MLP remain FP32. The resulting firmware is therefore a mixed-precision storage implementation, not a fully integer inference path. Quantization-aware training is not used.

6.1. Per-row symmetric quantization

For row r of either recurrent matrix, the implementation computes

$$s_r = \max\left(\frac{\max_h |w_{r,h}|}{127}, \varepsilon_q\right), \tag{18}$$

$$q_{r,h} = \mathrm{clip}\left(\mathrm{round}\left(\frac{w_{r,h}}{s_r}\right), -127, 127\right), \tag{19}$$

$$\hat{w}_{r,h} = s_r q_{r,h}, \tag{20}$$

where ε_q prevents division by zero for an all-zero row. Although the INT8 storage type provides 256 machine codes, the implementation deliberately leaves -128 unused and employs the symmetric 255-level set $\{-127, \ldots, 127\}$. This keeps zero exactly representable and assigns equal positive and negative ranges. With the max-absolute scale in Eq. (**??**), finite row weights do not clip, and nearest-integer rounding gives the element-wise reconstruction bound

$$|w_{r,h} - \hat{w}_{r,h}| \le \frac{s_r}{2}. \tag{21}$$

Per-row scaling gives small-magnitude rows a finer step size than one global scale, following standard uniform post-training quantization practice [**? ?**].

6.2. Mixed-precision gate computation

Let r denote one gate row. The firmware evaluates

$$g_r = \sum_{d=1}^{D} x_d (s_{\mathrm{ih},r} q_{\mathrm{ih},r,d}) + \sum_{j=1}^{H} h_j (s_{\mathrm{hh},r} q_{\mathrm{hh},r,j}) + b_{\mathrm{fp32},r}. \tag{22}$$

Thus the recurrent weights are stored as INT8 codes but reconstructed into FP32 products inside the current accumulation loops. A built-in streaming comparison validates the exported gate computations against the source model and typically yields MAE differences below 10^{-4}.

The recurrent matrices account for approximately 80 % of the raw Base model constants in both tasks, which motivated targeting them while leaving the smaller MLP path unchanged. This choice reduces persistent storage but preserves FP32 arithmetic and recurrent-state RAM. It therefore cannot provide the latency and RAM advantages of a fully integer path. Activation quantization was not evaluated, and the accuracy contribution of retaining FP32 activations was not isolated experimentally. **??** documents the verified precision boundary and model-constant composition.

6.3. Export for STM32

The exported headers are integrated into separate C firmware projects for the stateful SOC and SOH estimators. Each project combines the LSTM and MLP kernels with UART telemetry. A small driver library provides build files and flashing scripts so that identical measurement binaries can be rebuilt on other machines. This explicit tool chain, from training through pruning and quantization to firmware, exposes the numerical operations and records the build settings used for the resource measurements in Section **??**.

7. Benchmark Methodology

This section describes how we benchmark the SOC and SOH models on both the host PC and an STM32 Nucleo-144 development board with an STM32H753ZI MCU [**? ?**]. The microcontroller provides 2 MB of on-chip Flash and 1 MB of SRAM, which constitute the budget for firmware code, model parameters, and runtime buffers. Starting from the trained LSTM + MLP models, we generate three embedded variants (full-precision baseline, pruned model, and quantized model) and evaluate them on the *LFP DoE Cycle Ageing Dataset*. The benchmark combines long-horizon streaming simulations with hardware-in-the-loop measurements and is fully scripted to ensure reproducible accuracy and resource evaluations.

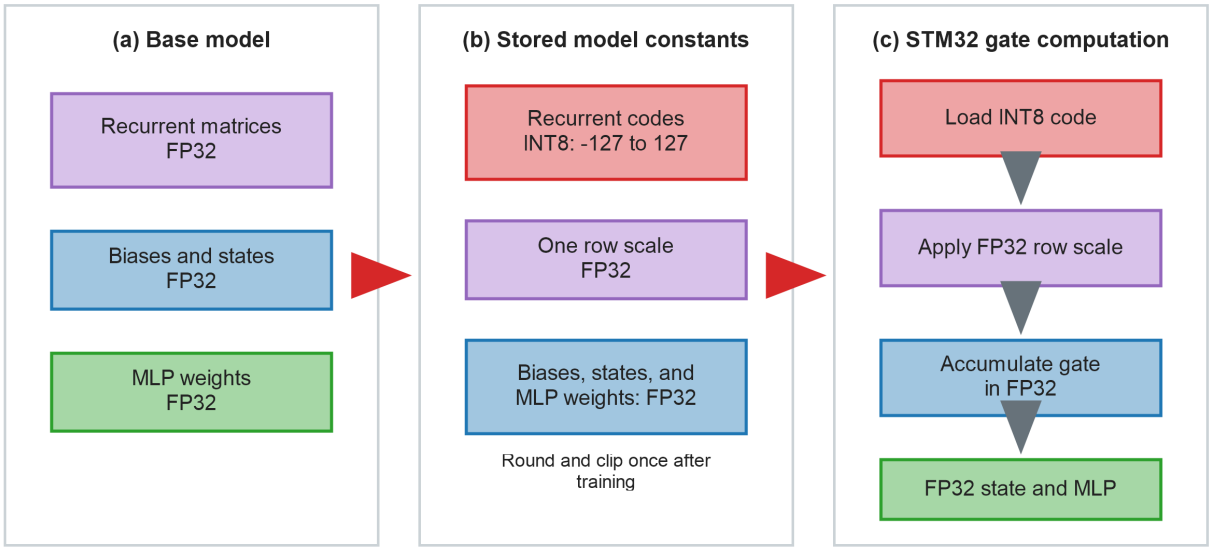


Figure 7: Implemented weight-only mixed-precision path. Panel (a) shows the FP32 Base constants. Panel (b) shows the stored representation after per-row symmetric quantization: recurrent codes use the set $\{-127, \ldots, 127\}$, whereas row scales, biases, states, and MLP weights remain FP32. Panel (c) shows that the STM32 kernel loads each INT8 code, applies its FP32 row scale, and accumulates the gate in FP32 before the FP32 recurrent-state and MLP operations.

On the host side, the complete reference ageing sequence is replayed in Python using the same streaming inference functions that are later exported to C. The SOC and SOH estimators are evaluated in continuous time without resetting their recurrent states, and error metrics (MAE, RMSE, R^2), error histograms, and parity plots are computed from the resulting trajectories. These simulations provide a high-precision reference for the embedded experiments and are used to assess how pruning and quantization affect the dynamics of the baseline models. To test whether deviations grow over time, the saved full-stream outputs are additionally divided into ten consecutive, equal-count segments. Target MAE and P95 error are computed in every segment, together with the mean absolute deviation of each compressed variant from the Base output; **??** reports this temporal analysis.

For the embedded benchmarks the base, pruned, and quantized SOC/SOH models are embedded into dedicated firmware projects for the STM32H753ZI. Because STM32Cube AI does not natively support stateful LSTM layers, the recurrent update equations, post-processing, and UART diagnostics are implemented manually in C. Identical stimulus streams are then injected via UART, allowing a direct comparison between host-side and on-device behaviour under the same operating conditions.

7.1. Limited software-level fault sensitivity

Two post-hoc checks are applied to the saved output trajectories without rerunning the microcontroller. First, randomly omitted reported outputs at rates of 1, 5, and 10 %, as well as ten deterministic gaps of 60, 600, or 3600 samples, are replaced by the preceding prediction (hold-last). Second, for each task and model, 20,000 trials randomly select one FP32 output value and flip either a mantissa, exponent, sign, or arbitrary bit. A basic guard accepts only finite outputs in the physical interval $[0, 1]$ and otherwise holds the preceding prediction. We report the fraction of corrupted values with an absolute error above 10 percentage points and the P95 absolute error before and after this guard. This is deliberately a limited software sensitivity analysis: it does not inject faults into weights, activations, recurrent states, sensor inputs, communication packets, or MCU memory, and it does not model state evolution when inference itself is interrupted.

7.2. STM32 Measurements

On the STM32 Nucleo-144 development board, dedicated SOC and SOH firmware images execute the embedded estimators while host-side scripts handle stimulus injection and UART logging. Recorded feature streams are trans-

Table 1: Engineering KPIs for SOC models on STM32H753ZI (full-stream averages). Energy is estimated from measured inference time under an assumed constant board power of 0.5 W.

Model	Latency [ms]	Inference [ms]	Flash [KB]	RAM [KB]	E_{est} [mJ]
Base FP32	12.70	1.40	105.32	4.93	0.70
Pruned FP32	12.20 (-3.9%)	0.80 (-42.9%)	62.27 (-40.9%)	4.03 (-18.3%)	0.40 (-42.9%)
Quant INT8	18.48 (+45.5%)	6.99 (+399%)	52.48 (-50.2%)	3.96 (-19.7%)	3.49 (+399%)

mitted to the microcontroller, which runs the stateful LSTM + MLP and returns predictions with timing diagnostics.

Each sample produces two time stamps. Host latency measures the delay between sending an input vector and receiving its estimate. On-device inference time is measured with the data watchpoint and trace (DWT) cycle counter around the LSTM and MLP kernels. Reported values are averages over 10,000 streaming inferences. The timed interval includes instructions and memory accesses executed by these kernels but excludes UART transfer, host processing, and host-side feature construction.

In the SOC projects, six-feature robust scaling lies inside the timed estimator call. The SOH Base and Pruned projects apply scaling immediately before the DWT interval, whereas the Quantized SOH forward function includes it. This small affine preprocessing cost was not profiled separately and is absent from the static operation counts. All inspected benchmark projects were compiled with GCC at `-O0`, using the hard-float ABI and Cortex-M7 FPv5 floating-point unit. The measurements therefore characterise auditable, unoptimised reference kernels rather than production-optimised integer deployments.

7.3. Key Performance Indicators

For normalised target y_n and prediction $\hat{y}_n$, the signed error is expressed in percentage points as $e_n = 100(\hat{y}_n - y_n)$. The reported accuracy indicators are

$$\text{MAE} = \frac{1}{N}\sum_{n=1}^{N}|e_n|, \qquad \text{RMSE} = \sqrt{\frac{1}{N}\sum_{n=1}^{N}e_n^2}, \tag{23}$$

$$\text{P95} = Q_{0.95}\Big(\{|e_n|\}_{n=1}^{N}\Big), \tag{24}$$

where $Q_{0.95}$ is the empirical 95th percentile. MAE and RMSE use all samples, whereas P95 identifies the absolute-error threshold below which 95 % of samples lie.

For each deployment we additionally report average host latency, on-device inference time, flash and SRAM usage, and an estimated energy-per-inference proxy. Tables **??** and **??** focus on these STM32 resource and timing indicators. Latency is the end-to-end UART delay, whereas inference time covers only the DWT-measured kernel interval. RAM combines statically allocated data and BSS sections with maximum observed streaming stack usage.

Energy was not measured separately for each model. Instead, we use

$$E_{\text{est}} = P_{\text{assumed}}\, t_{\text{inf}}, \qquad P_{\text{assumed}} = 0.5\,\text{W}, \tag{25}$$

where 0.5 W is a representative average-power assumption for the complete development board. Consequently, the percentage change in E_{est} is identical to the percentage change in inference time. The proxy does not separate core computation, flash access, SRAM access, voltage-regulator losses, or other board consumers and cannot determine whether flash access dominates the actual energy consumption. An analogous set of measurements was recorded for the SOH firmware deployments. Table **??** lists the corresponding indicators.

The KPIs in Tables **??** and **??** highlight the trade-offs introduced by pruning and quantization. For SOC, pruning reduces the flash footprint by roughly 41 % and inference time by about 43 %, while latency remains dominated by UART transfer. Quantization approximately halves flash usage, but per-weight scaling and integer–float conversions in the mixed-precision kernels increase on-device inference time by a factor of four relative to Base. The SOH deployments show the same qualitative behaviour: pruning saves about 46 % of flash and 44 % of inference time, whereas Quantized attains the smallest flash footprint but incurs a 29 % timing increase. Under the constant-power assumption, the estimated energy proxy follows these timing changes exactly; it is not independent evidence of board-level energy efficiency.

Table 2: Engineering KPIs for SOH models on STM32H753ZI (full-stream averages). Energy is estimated from measured inference time under an assumed constant board power of 0.5 W.

Model	Latency [ms]	Inference [ms]	Flash [KB]	RAM [KB]	E_{est} [mJ]
Base FP32	34.88	22.73	335.00	8.69	11.37
Pruned FP32	24.69 (-29.2%)	12.72 (-44.0%)	182.41 (-45.5%)	6.96 (-19.9%)	6.36 (-44.0%)
Quant INT8	41.23 (+18.2%)	29.21 (+28.5%)	138.00 (-58.8%)	6.70 (-22.9%)	14.60 (+28.5%)

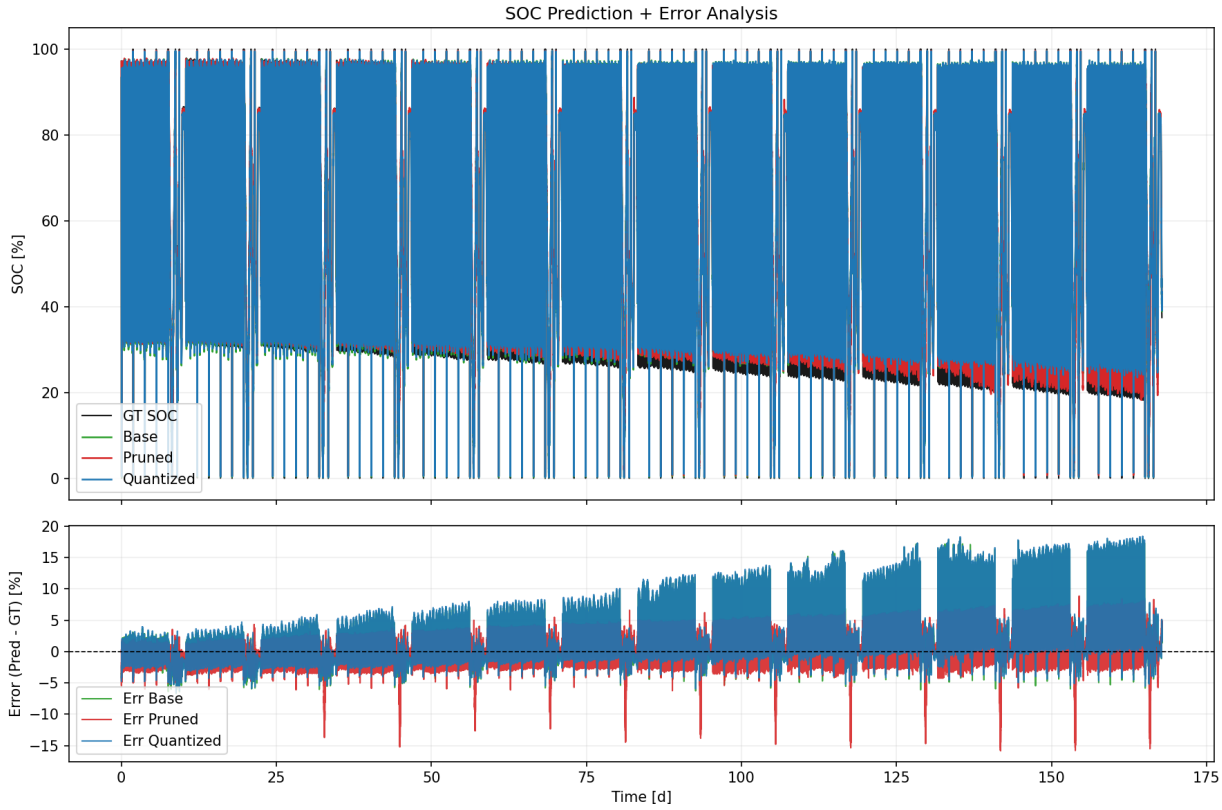


Figure 8: Streaming SOC dashboard on the full ageing trajectory. The black line represents the reference SOC, while the coloured lines show the predictions of the Base (green), Pruned (red), and Quantized (blue) models. The lower sub-panel displays the instantaneous prediction error.

8. Results

The evaluation compares the full-precision Base, Pruned, and Quantized SOC and SOH estimators under the same continuous streaming replay of the LFP DoE Cycle Ageing Dataset. The following sections separate prediction accuracy, temporal behaviour, embedded resource use, and the resulting multi-objective trade-offs.

8.1. Streaming Prediction Accuracy

The primary metric for any battery state estimator is its ability to track the ground truth over extended periods without divergence. Figures **??** and **??** illustrate the streaming performance of the SOC and SOH models over the complete test duration. Across the complete SOC replay, the Base, Pruned, and Quantized variants obtain MAE values of 2.685, 2.338, and 2.791 percentage points, respectively. The Quantized trace often overlaps the Base trace visually, whereas the Pruned checkpoint has the lowest aggregate target MAE despite a larger model-to-Base deviation in some parts of the sequence. These results show that the evaluated post-training weight quantization preserves useful recurrent behaviour on this test stream; they do not establish identical internal states or transfer to another sequence.

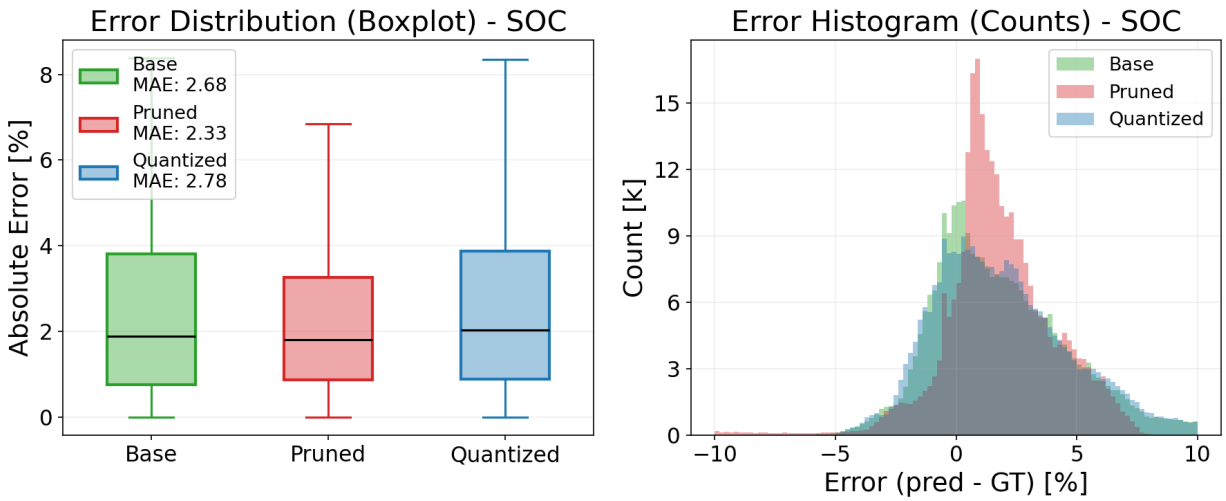


Figure 9: SOC error distributions for Base, Pruned, and Quantized. The left panel compares absolute errors and aggregate MAE; the right panel shows signed errors, $100(\hat{y} - y)$, so that distribution shifts and tails remain visible.

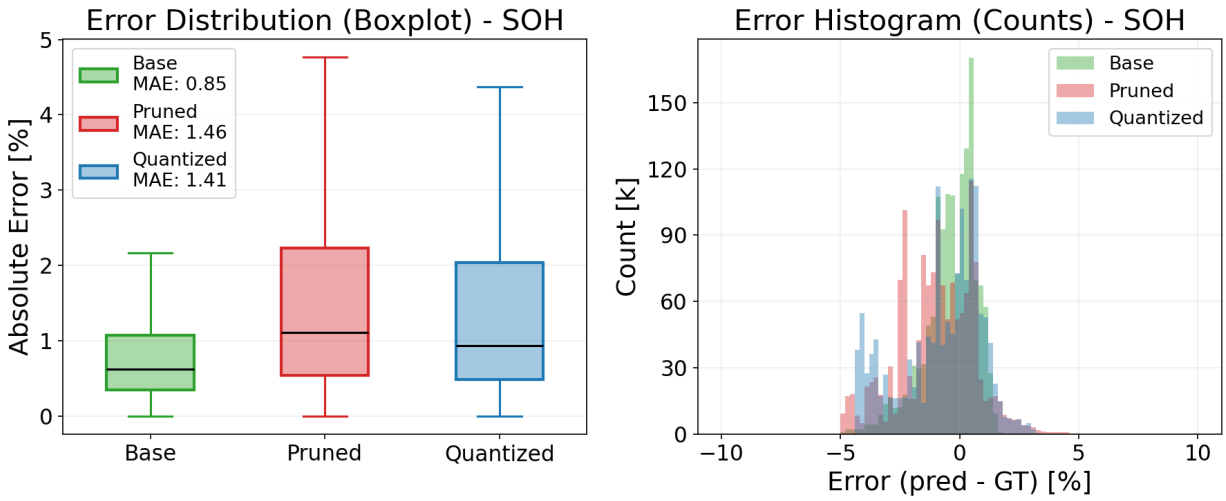


Figure 10: SOH error analysis: MAE distribution (left) and error histogram (right). The three variants retain similar central error ranges, while their tails and aggregate MAE values differ.

The SOH estimator faces a different challenge: it must infer slow capacity degradation from sparse check-up events and continuous monitoring. In the plotted trajectories the discrete capacity tests have been removed from the estimator input, while the remaining check-up blocks (OCV sweeps, HPPC pulses, and profile segments) are retained. The plateau-like sections of the reference curve therefore correspond to these diagnostic intervals. The dashboard applies the complete two-stage causal post-processing chain from Section **??**: initial alignment, stage 1 with $\alpha_1 = 0.02$ and a symmetric limiter, followed by stage 2 with $\alpha_2 = 10^{-6}$ and a downward limiter. The resulting smoothing suppresses short-term variation but introduces a substantial accuracy–delay trade-off.

8.2. Temporal stability and filter interaction

The ten-segment analysis in **??** retains the natural temporal order and does not reset recurrent states between segments. SOC target error rises toward the later part of the sequence for every variant: for example, Base MAE

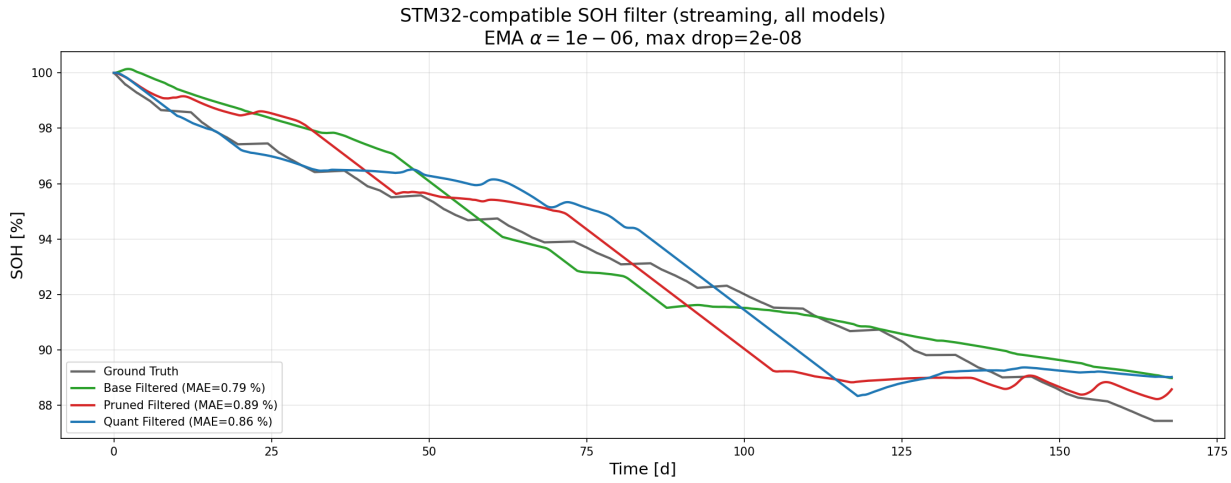


Figure 11: Streaming SOH dashboard with the complete two-stage causal post-processing chain from Section **??**. The top panel shows the reference SOH and the outputs of the Base (green), Pruned (red), and Quantized (blue) models; the bottom panel depicts the corresponding filtered prediction errors.

increases from 1.116 to 4.538 percentage points and Quantized MAE from 1.234 to 4.600 percentage points between the first and final tenths. However, the Quantized-to-Base mean absolute deviation changes from 0.332 to 0.271 percentage points over the same interval and therefore shows no quantization-specific monotonic accumulation. The Pruned-to-Base deviation increases from 0.924 to 2.592 percentage points even though Pruned has the lowest aggregate target MAE. SOH window errors fluctuate with the ageing phase, and neither compressed variant exhibits monotonic model-to-Base divergence across all ten segments. This supports stability for the recorded replay, not a general proof for arbitrarily long operation.

?? compares unfiltered, stage-1, and final two-stage SOH outputs for all three exported C models in one local re-execution. Stage 1 changes the MAE to 1.677, 1.644, and 1.926 percentage points for Base, Pruned, and Quantized, respectively; applying stage 2 to the stage-1 output gives final MAE values of 1.476, 1.695, and 1.028 percentage points. Thus, compression and filtering cannot be interpreted independently, and the lowest MAE does not by itself imply the best dynamic response. The final EMA has an EMA-only 90 % response time of 26.65 days at 1 Hz. Because this local Windows re-execution is not numerically identical to the archived Linux trajectory over the complete recurrent sequence, it is used only for the controlled filter comparison and does not replace the main reported benchmark values.

8.3. Error Distribution Analysis

To quantify the reliability of the estimators, we analyse the statistical distribution of the prediction errors. Figures **??** and **??** present the Mean Absolute Error (MAE) boxplots and the error histograms for both tasks. The SOC error distribution is narrow and approximately centred around zero. The evaluated Pruned checkpoint attains the smallest aggregate MAE, while the Quantized model remains close to the Base result with a smaller recurrent-weight footprint. **??** shows that the 19 removed SOC units occupy the lowest positions of the defined L2 saliency ranking and compares the central recurrent-weight distributions before and after pruning. This provides descriptive evidence that the implemented criterion removed the intended low-saliency channels. Because only one trained checkpoint and test split were evaluated, the small MAE improvement cannot be attributed causally to regularisation and may depend on model initialisation or data partitioning. For SOH, the error distribution is broader, which is inherent to the difficulty of estimating capacity fade from operational data. However, the histograms show that the bulk of the errors are still contained within a useful tolerance band (typically ±2%), making the embedded estimator a viable tool for onboard diagnostics.

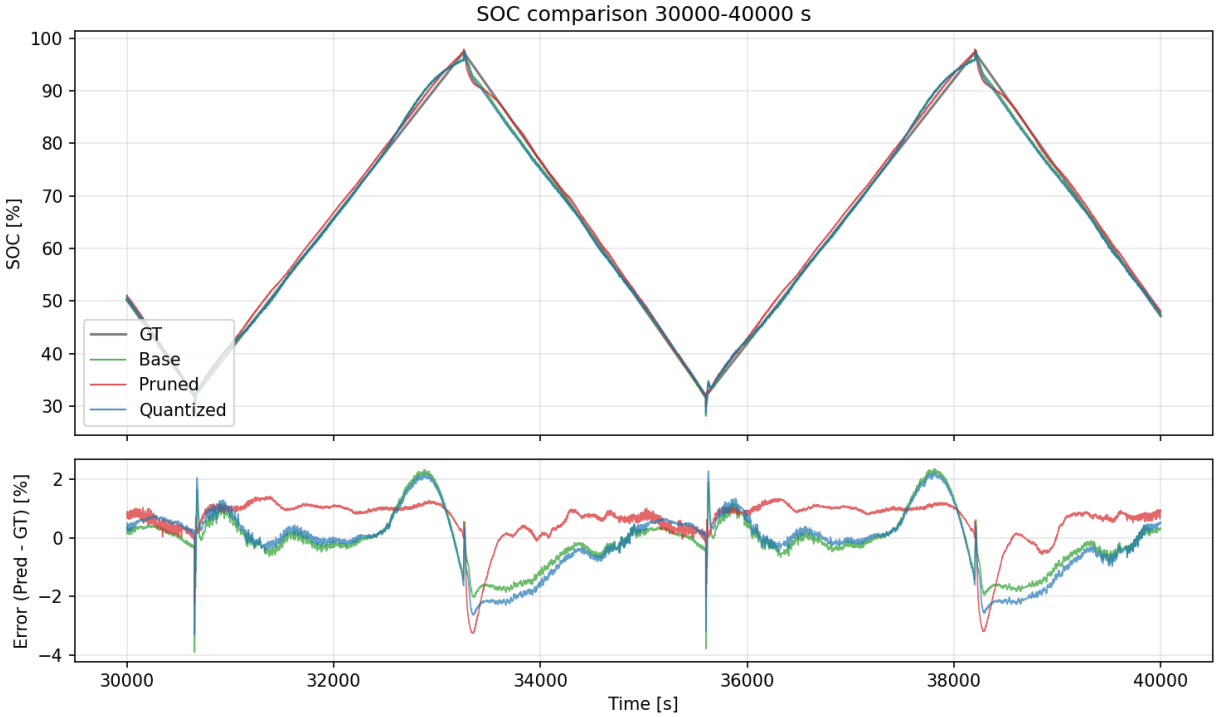


Figure 12: Detailed transient analysis of SOC streaming replay (pulse segment). Response to dynamic load pulses (30k–40k s).

8.4. Transient Response and Zoomed Analysis

Global metrics often hide local failures. To inspect the transient behaviour, we zoom into a high-dynamic pulse discharge phase (Fig. **??**) and into a representative check-up sequence (Fig. **??**).

In the pulse region (Fig. **??**), the models must react to rapid current changes. The quantized model follows the ground truth transients with high fidelity, capturing the voltage recovery phases accurately. The pruned model, having fewer parameters to model the complex electrochemical relaxation, tends to smooth out these sharp features slightly. During the check-up phase (Fig. **??**), where the cell undergoes full charge/discharge cycles, all models converge well, demonstrating stability over long integration horizons.

8.5. Resource Efficiency and Latency

Figures **??** and **??** summarise the flash memory, RAM usage, parameter counts, and latency distributions for the SOC and SOH deployments on the STM32H7 microcontroller. For all deployments the model complexity is first characterised on the workstation side by counting parameters in each variant (base, pruned, quantized). Assuming four bytes per weight for floating point representations and one byte per weight for INT8, this directly yields an estimate of the flash footprint attributable to the neural network weights alone. The second stage of the analysis uses the linker map files of the STM32CubeIDE projects to determine the actual binary size in flash, which also includes code, constant tables, and run-time support libraries. This explains why the measured flash usage in Figure **??** is systematically higher than the idealised "weights only" estimate, and why the gap is larger for the more complex SOH firmware that contains additional diagnostic and filtering logic. The RAM usage shown in the lower right panel of Figure **??** is obtained from on-device measurements of the SOH and SOC firmware and applied analogously to all STM32 deployments. At boot time the firmware reports the size of the statically allocated data and BSS sections, which correspond to persistent variables and network parameters stored in SRAM. During each benchmark inference the minimum value of the stack pointer is tracked, so that the maximum additional stack consumption of the call chain can be recovered. The total RAM requirement reported in the figure is the sum of static data and

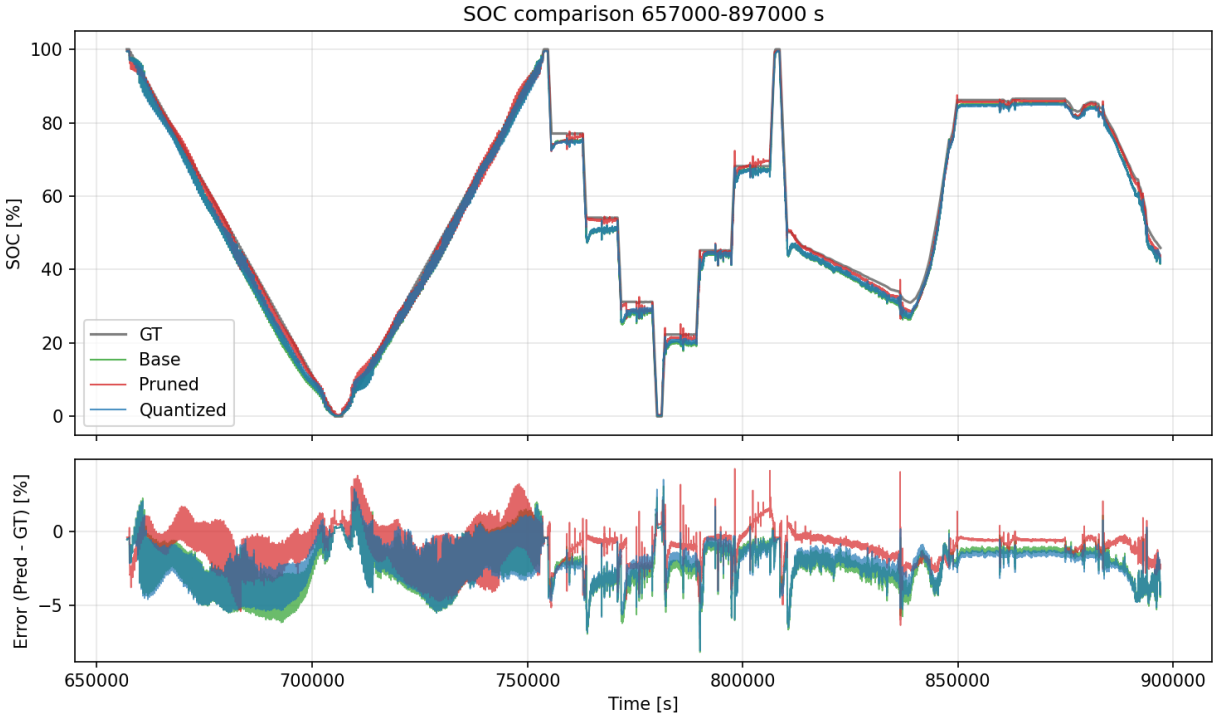


Figure 13: Detailed transient analysis of SOC streaming replay (check-up segment). Behaviour during a capacity check-up (657k–897k s).

worst-case stack usage, i.e. an upper bound on the memory needed to execute one inference including the LSTM state updates and UART handling. This procedure captures the fact that pruning reduces both the number of parameters and the recurrent state dimension, whereas the weight-only quantized implementation primarily reduces recurrent-matrix storage. Activations, hidden and cell states, biases, and the MLP remain FP32. Figure **??** shows that Quantized produces the smallest flash footprint, while Pruned reduces both the parameter count and recurrent-state dimension. The measured RAM and execution-time results must nevertheless be considered separately because retaining FP32 activations and changing the kernel arithmetic prevents storage reduction from translating automatically into lower latency.

Figure **??** reports the corresponding host-side latency distributions. The latency distributions in Figure **??** reveal a distinct hierarchy. SOC inference requires approximately 0.8 ms to 7 ms, whereas the SOH variants require 12 ms to 30 ms per update. The weight-quantized SOC deployment is slowest because the hand-written mixed-precision kernel applies FP32 row scales during the recurrent matrix products. When UART framing and host processing are included, end-to-end latencies remain below roughly 20 ms for SOC and 50 ms for SOH.

8.6. Composite trade-off summary

To make the multi-objective trade-offs easier to interpret at a glance, we report a composite utility score U that aggregates accuracy and embedded resource indicators into a single, dimensionless index. For task-specific non-negative weights $\mathbf{w} = (w_A, w_F, w_R, w_E)$ with $\sum_j w_j = 1$, model variant v is scored as

$$U_v(\mathbf{w}) = w_A \frac{\mathrm{MAE}_v}{\mathrm{MAE}_{\mathrm{Base}}} + w_F \frac{F_v}{F_{\mathrm{Base}}} + w_R \frac{R_v}{R_{\mathrm{Base}}} + w_E \frac{E_{\mathrm{est},v}}{E_{\mathrm{est,Base}}}, \tag{26}$$

where MAE is the streaming mean absolute error, F is measured flash, R is measured total RAM (static plus peak stack), and E_{est} is the timing-derived energy proxy. The main comparison uses equal weights, $w_A = w_F = w_R = w_E = 0.25$, which reproduces the arithmetic mean used in the original score. By construction $U_{\mathrm{Base}} = 1$ for every

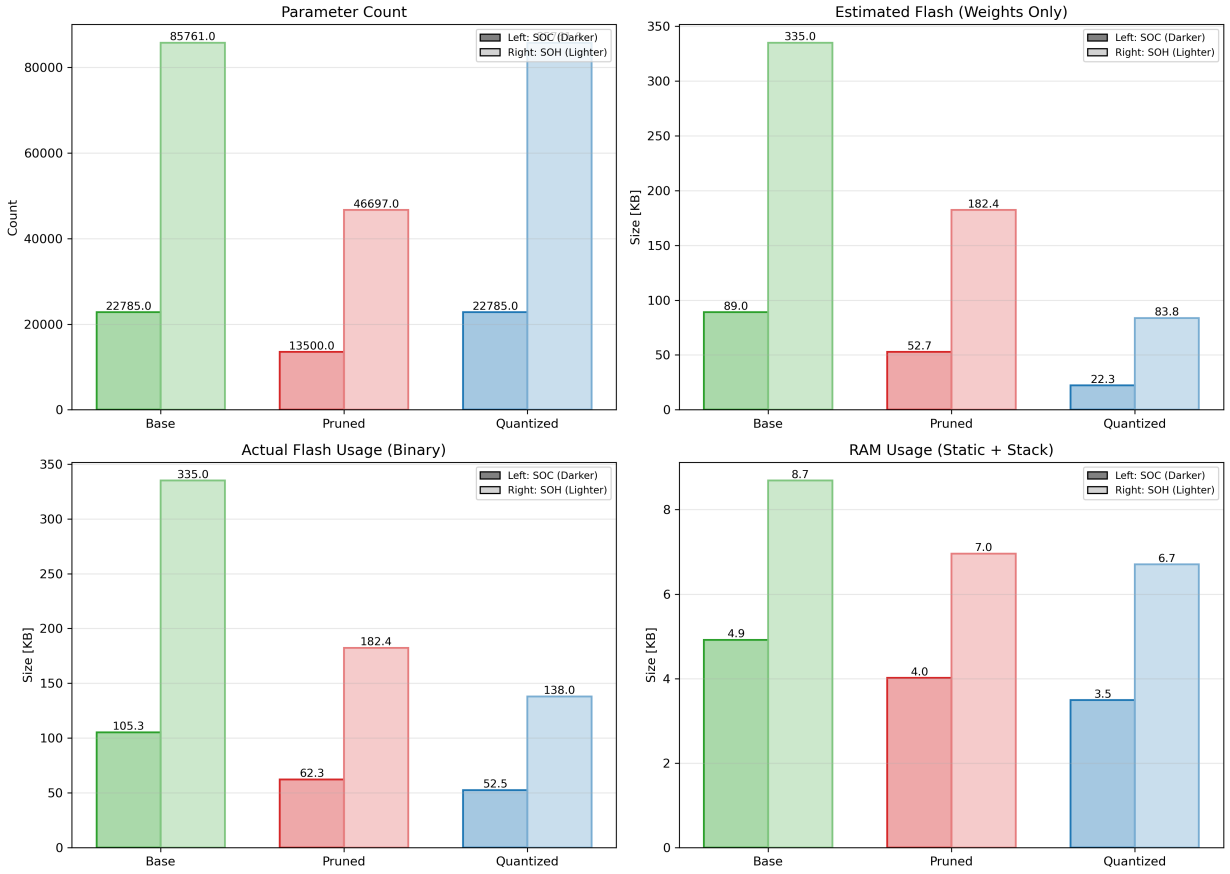


Figure 14: Resource landscape for SOC and SOH models. Comparison of parameter count, estimated Flash, actual Flash, and RAM usage. Dark bars denote SOC, light bars denote SOH.

Table 3: At-a-glance effect of pruning and quantization relative to the corresponding Base model. Values are reported as SOC/SOH. ΔMAE is reported in percentage points (pp) of full scale (negative indicates improved accuracy); flash, RAM, and estimated-energy changes are relative deltas versus Base. The composite score U uses equal weights in (**??**) (lower is better).

Variant	ΔMAE [pp]	ΔFlash [%]	ΔRAM [%]	ΔE_{est} [%]	U
Pruned	-0.35/+0.61	-41/-46	-18/-20	-43/-44	0.71/0.91
Quantized	+0.10/+0.56	-50/-59	-20/-23	+399/+29	1.83/1.03

valid weight vector; lower values are preferred. Because E_{est} is proportional to inference time under the fixed-power assumption, the energy term represents timing priority rather than an independent power measurement.

Sensitivity is evaluated over all 1,771 non-negative combinations on a 0.05 grid that sum to one. **??** also varies one highlighted objective from 25 to 85 %, sharing the remaining weight equally among the other three. Pruned is top-ranked for 94.64 % of SOC combinations and 53.36 % of SOH combinations. Quantized wins 5.36 % and 17.79 %, respectively, while Base wins 0 % for SOC and 28.85 % for SOH. The ranking is therefore robust for SOC Pruned but application-dependent for SOH: Base becomes preferable when accuracy dominates, whereas Quantized becomes preferable when flash receives high weight. Table **??** summarises the resulting changes for pruning and quantization; values are reported as SOC/SOH.

8.7. Limited output-fault sensitivity

?? summarises the software-level bit-flip experiment. For an arbitrary bit position, 33.1–34.3 % of trials produce errors above 10 percentage points across the six task/model combinations. Rejecting non-finite or out-of-range outputs

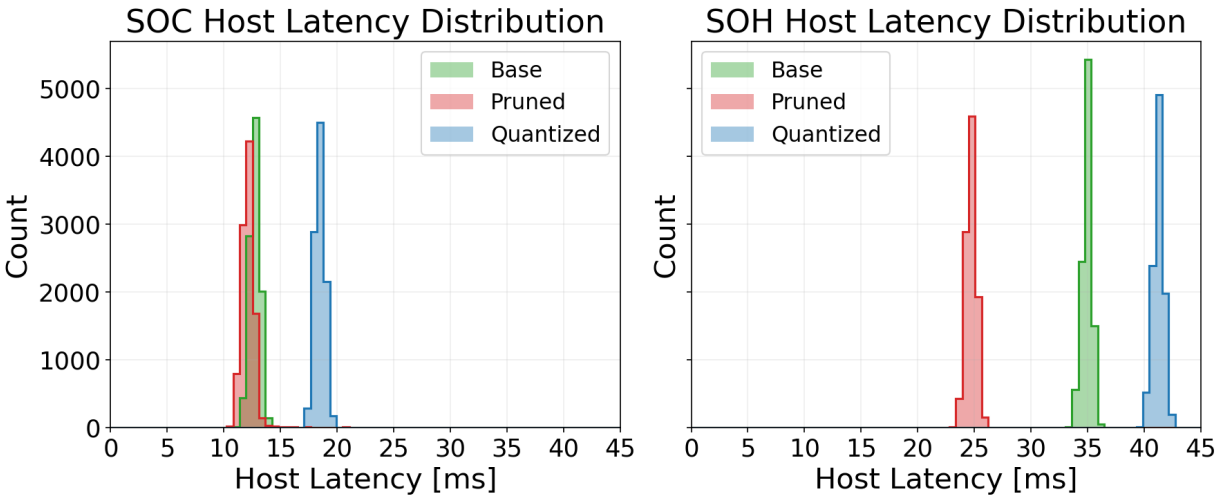


Figure 15: Latency distributions for SOC and SOH models on the STM32H7. Histograms are computed from repeated on-device measurements.

and holding the preceding prediction reduces the P95 absolute error, but it remains approximately 72–73 percentage points for SOC and 94 percentage points for SOH because physically plausible in-range corruptions are not detected. In the separate missing-update check, holding the preceding saved output for up to 10 % randomly omitted reports changes aggregate MAE by less than 0.001 percentage points. Ten gaps of 3600 samples increase SOC MAE by approximately 0.05 percentage points and have negligible influence on the slowly varying saved SOH output. These results demonstrate the value and the clear insufficiency of a range check; they are not internal MCU or recurrent-state fault-injection results.

9. Discussion

The experiments show that pruning and quantization affect the SOC and SOH estimators differently. For SOC, the evaluated Pruned checkpoint obtains a 0.35-percentage-point lower MAE than Base, while Quantized remains 0.11 percentage points above Base. The L2 saliency and weight-distribution diagnostics confirm which channels were selected, but they do not show that pruning caused regularisation or improved generalisation. No repeated-seed, cross-validation, or matched unpruned fine-tuning experiment was available; the lower Pruned MAE is therefore reported as a checkpoint- and split-specific observation.

For SOH the picture is more nuanced. Targets evolve slowly and are anchored by periodic capacity check-ups, while the displayed online estimates undergo two strong filtering stages. The local filter comparison shows that post-processing can alter not only absolute MAE but also the ranking between Base, Pruned, and Quantized. Compression quality should therefore be assessed on both raw and filtered outputs, and filter selection should consider response delay rather than MAE alone.

From an engineering perspective the three deployment variants represent distinct operating points. Base FP32 offers the simplest tool flow but requires the largest flash footprint. Pruning reduces the recurrent state dimension and the corresponding MLP input width, shrinking SOC flash from 105.32 to 62.27 kB and inference time from 1.40 to 0.80 ms. For SOH the corresponding changes are 335.00 to 182.41 kB and 22.73 to 12.72 ms. Within this unchanged FP32 kernel family, static MAC ratios and measured time ratios agree closely: SOC Pruned retains 54.9 % of Base MACs and 57.2 % of its time, while SOH retains 54.3 % and 56.0 %, respectively.

Weight-only quantization gives the smallest flash footprints, 52.48 kB for SOC and 138.00 kB for SOH, but changes the operation types without providing a fully integer execution path. Base and Quantized retain the same 22,080 SOC and 85,120 SOH model MACs. In the current C expressions, each recurrent INT8 weight is cast to FP32 and multiplied by its row scale inside the innermost accumulation, adding 17,920 source-level FP32 scale multiplications for SOC and 68,608 for SOH. Treating one MAC and one additional multiplication as one operation

each gives a static Quantized/Base ratio of approximately 1.81 for both tasks, whereas the measured time ratios are 4.99 for SOC and 1.29 for SOH.

The discrepancy is expected because the count omits conversion, indexing, loop, load/store, nonlinear-function, and cache/Flash effects and because the `-O0` kernel uses no optimised integer dot-product path. For the larger SOH model, reduced recurrent-weight traffic may offset more of this arithmetic overhead than for SOC. Memory bandwidth was not measured, so this remains an implementation-based interpretation rather than a causal attribution. **??** places the static counts beside the measured DWT times.

Possible kernel improvements include accumulating each row before applying its shared scale, fusing scale and bias work, using DSP-optimised or integer dot-product kernels, and quantizing activations and recurrent states to enable a complete integer path. These options were not implemented. No operation-level cycle breakdown or memory-bandwidth profile is available, so the present results establish total kernel timing for the tested firmware but not the fraction caused by conversion, arithmetic, or memory access.

The distinction between system latency, inference time, and estimated energy is particularly relevant for resource-constrained BMS. The UART-driven tests report end-to-end latency, while on-device inference spans 0.80–6.99 ms for SOC and 12.72–29.21 ms for SOH. The reported E_{est} values are obtained by multiplying these times by an assumed constant 0.5 W; they do not isolate CPU, flash, SRAM, UART, or regulator contributions. Consequently, the results establish timing and memory trade-offs on the STM32H753ZI, but not a component-level energy budget or a conclusion that flash access dominates consumption.

More generally, the reported measurements describe a multi-objective trade-off space rather than a single "best" solution. The equal-weight score gives $U = 0.71$ for SOC Pruned and $U = 1.83$ for SOC Quantized, whereas both compressed SOH variants remain near one. The full weight-grid analysis shows why these scalar values must not be interpreted as universal rankings: SOC Pruned is stable across most priorities, but the preferred SOH model changes when accuracy or flash dominates. In practice, hard accuracy and memory constraints should be applied before a scalar utility score is used.

The implementation-level storage reduction is not intrinsically tied to LFP chemistry: removing the same number of hidden channels or storing the same recurrent matrices as INT8 produces similar parameter-count changes for a fixed architecture. The accuracy–compression trade-off is chemistry- and data-dependent, however. This study trains and evaluates only JGC LFP cells at 25 °C; it does not validate NMC or LCO voltage characteristics, degradation trajectories, or operating ranges. Existing embedded NMC SOC studies and NMC SOH estimators demonstrate that compact data-driven models are feasible on those chemistries [**? ? ?**], but they do not transfer the numerical MAE or optimal pruning/quantization level reported here. Chemistry-specific retraining and validation are required before those values can be generalised.

The present variants also separate the two compression routes: one-shot structured pruning with short fine-tuning, and post-training recurrent-weight quantization of the original FP32 checkpoint. Quantization-aware training and pruning-aware or iterative training can in principle recover accuracy by adapting parameters to the imposed representation [**? ?**], but were not run for this revision because they require a new training campaign. The current comparison therefore addresses deployable post-training transformations, not the best achievable result of jointly trained compression.

Current limitations are one LFP chemistry, one principal test split and trained checkpoint per task, one STM32 family, unoptimised reference builds, no quantization-aware or pruning-aware training, and no direct per-model power measurement. Hardware transfer depends on clock rate, FPU and DSP support, compiler optimisation, cache, and memory organisation. The analytical MAC trend therefore does not establish that the measured savings will persist on an STM32F4 or another controller.

The derivative features were prepared on the host. Consequently, variable sampling, missing measurements, derivative noise, and causal filtering or limiting were not evaluated as embedded modules in this compression benchmark. The long-horizon and output-fault analyses likewise use saved software trajectories. They do not cover recurrent-state, activation, weight-memory, sensor, communication, or MCU faults. The range-check experiment remains a basic sensitivity check rather than embedded safety validation.

10. Conclusion and Outlook

This study set out to quantify how two transparent compression routes change the accuracy and embedded cost of stateful SOC and SOH estimators under a common STM32 benchmark. Its contribution is an auditable comparison from saved LSTM + MLP checkpoints to continuous C inference, rather than a new recurrent architecture or compression algorithm. Estimated energy per inference remains a constant-power timing proxy and is not an independent power measurement.

For SOC, Pruned reduces flash by 40.9 %, RAM by approximately 18 %, and inference time by 42.9 %, while its MAE is 0.35 percentage points lower than Base for the evaluated checkpoint and split. Quantized reduces flash by 50.2 % and RAM by approximately 20 %, but increases inference time by 399 % and MAE by 0.11 percentage points in the mixed-precision software kernel. For SOH, Pruned reduces flash by 45.5 %, RAM by approximately 20 %, and inference time by 44.0 %, with a 0.61-percentage-point MAE increase. Quantized reduces flash by 58.8 % and RAM by approximately 23 %, while increasing inference time by 28.5 % and MAE by 0.56 percentage points. Under the fixed-power assumption, the energy proxy changes by the same percentages as inference time.

These results identify structured pruning as the more balanced operating point for the tested firmware, whereas recurrent-weight quantization is attractive primarily when flash capacity dominates and its dequantization overhead is acceptable. Ten consecutive replay segments show no quantization-specific monotonic deviation from Base. The SOH filter analysis further shows that compression cannot be assessed independently of post-processing delay. The conclusions are limited to the evaluated LFP trajectories, checkpoints, STM32H753ZI implementation, and compiler settings; they do not establish the same trade-off for NMC or LCO cells, other controller families, repeated model initialisations, or internal embedded faults.

The first priority for future work is to optimise and profile the mixed-precision kernel through row-scale hoisting, operator fusion, DSP-oriented dot products, and activation/state quantization, while measuring per-model board power with external instrumentation. A second priority is external validation across random seeds, temperatures, cell chemistries, and controller families, including direct comparison with quantization-aware and pruning-aware training. Finally, internal weight, activation, recurrent-state, sensor, communication, and MCU fault injection is required before safety-related claims can be made. Joint pruning and quantization remains a promising route for combining a reduced state dimension with compact recurrent storage.

Appendix A. Supplementary compression and robustness analyses

This appendix documents post-hoc analyses of the archived model artefacts and saved streaming outputs. Unless stated otherwise, no new STM32 measurement or neural-network training was performed.

Appendix A.1. Structured-pruning diagnostics

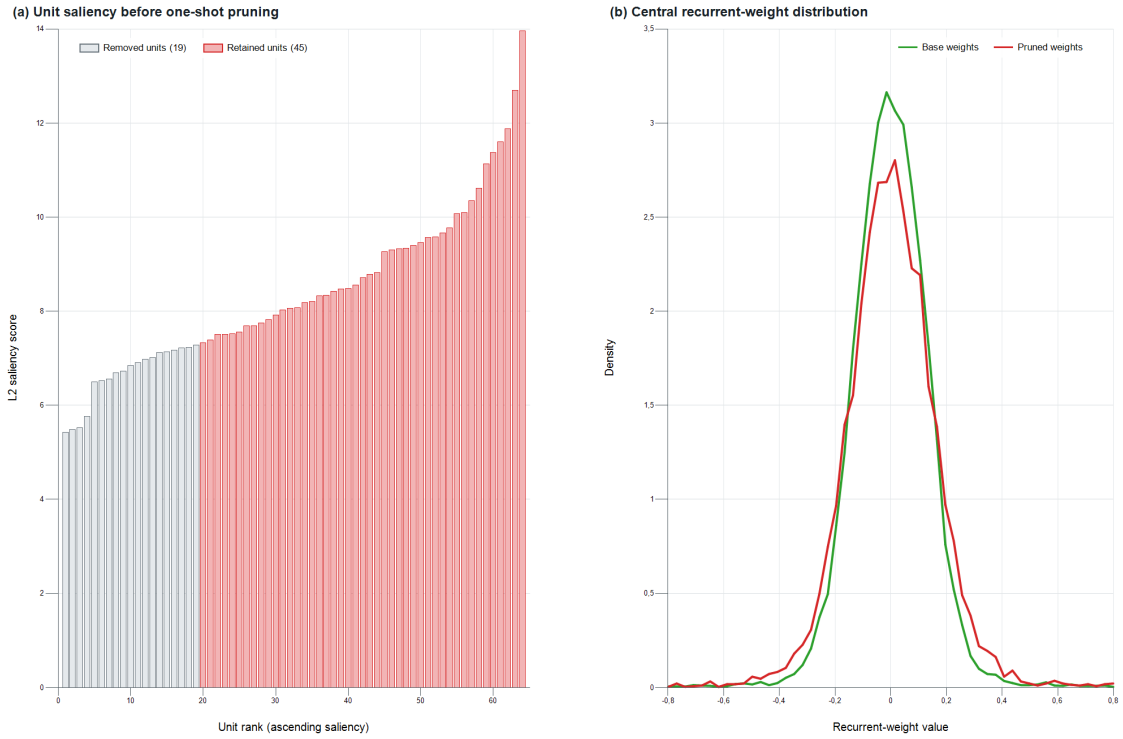


Figure A.16: Post-hoc diagnostics for the one-shot SOC pruning operation. (a) L2 saliency from Eq. **??**, sorted in ascending order; the 19 lowest-ranked units are removed and 45 units are retained. (b) Density of the central recurrent-weight range for the Base and Pruned checkpoints. The plot confirms application of the specified channel-ranking criterion and describes the resulting weight distribution. It does not prove that pruning caused the lower SOC MAE or that the effect persists across random seeds.

Appendix A.2. Long-horizon temporal stability

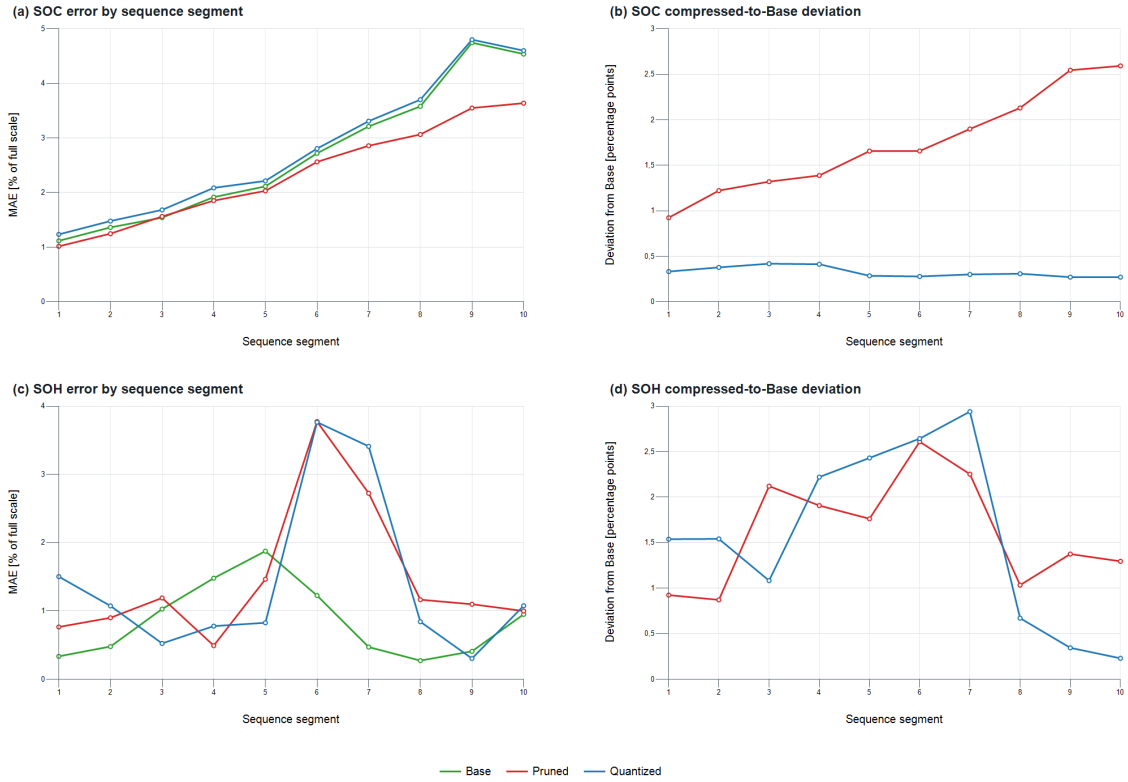


Figure A.17: Temporal error and model-to-Base deviation over ten consecutive, equal-count segments of the saved full-stream replay. Recurrent states are not reset at segment boundaries. Panels (a) and (c) show target MAE for SOC and SOH; panels (b) and (d) show the mean absolute deviation of Pruned and Quantized from Base. Quantized does not exhibit a monotonic late-sequence deviation from Base in either task, although target error varies with sequence phase. The result is specific to the recorded replay and is not a proof of stability for arbitrary sequence lengths.

Appendix A.3. SOH filtering and compression interaction

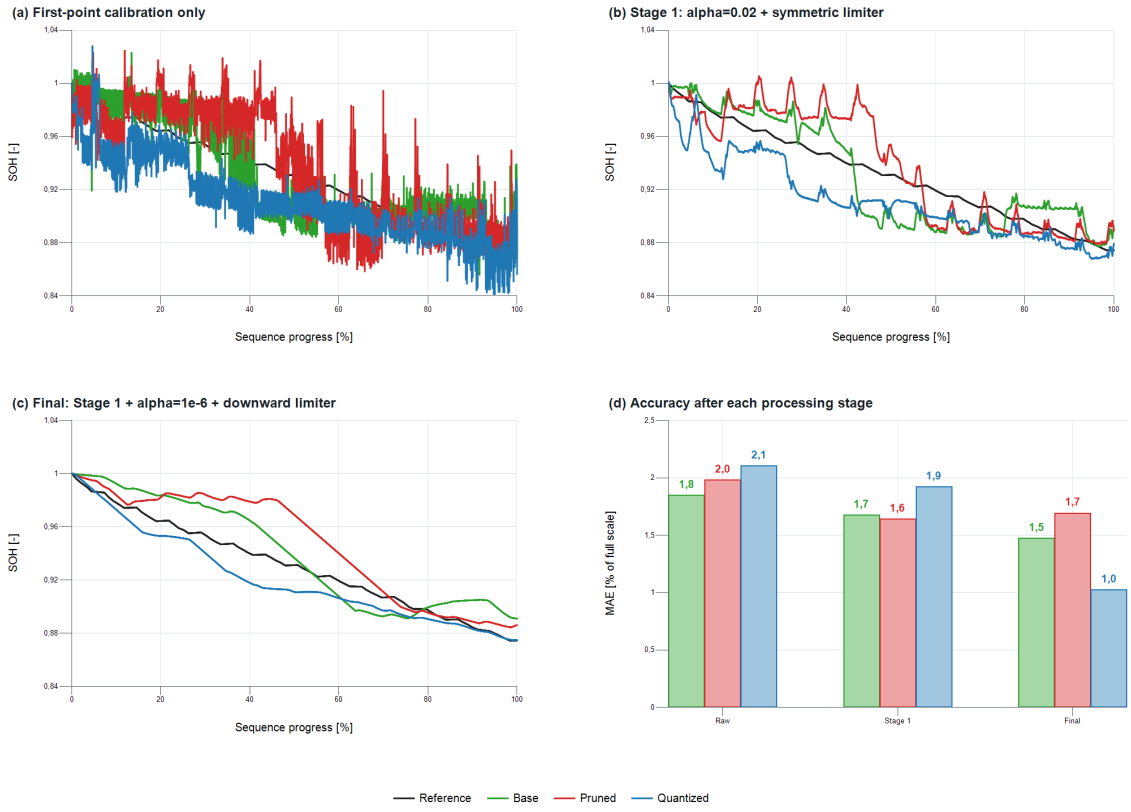


Figure A.18: Controlled comparison of SOH compression and filtering for locally re-executed Base, Pruned, and Quantized C models. (a) Outputs after first-point alignment but before temporal filtering. (b) Stage 1 with $\alpha_1 = 0.02$ and the symmetric limiter. (c) Final sequential output after applying stage 2 with $\alpha_2 = 10^{-6}$ and the downward limiter to stage 1. (d) MAE after each processing stage. At 1 Hz, the EMA-only 90 % response times are approximately 114 s and 26.65 days, respectively; the nonlinear limiters have no single cutoff frequency. The local Windows re-execution is internally consistent across the three variants but is not numerically identical to the archived Linux trajectory over the complete recurrent sequence, so these values analyse filter interaction and do not replace the main benchmark metrics.

Appendix A.4. Utility-weight sensitivity

Figure A.19: Sensitivity of the utility ranking to application priorities. Panels (a) and (b) vary the highlighted objective from 25 to 85 % and divide the remaining weight equally among the other three objectives; each square identifies the lowest-utility model. At 25 %, all four objectives have equal weight. Panel (c) gives the winning share across all 1,771 non-negative 0.05-spaced weight combinations that sum to one. The energy objective uses the timing-derived E_{est} proxy and is therefore not an independent power measurement.

Appendix A.5. Limited output-fault sensitivity

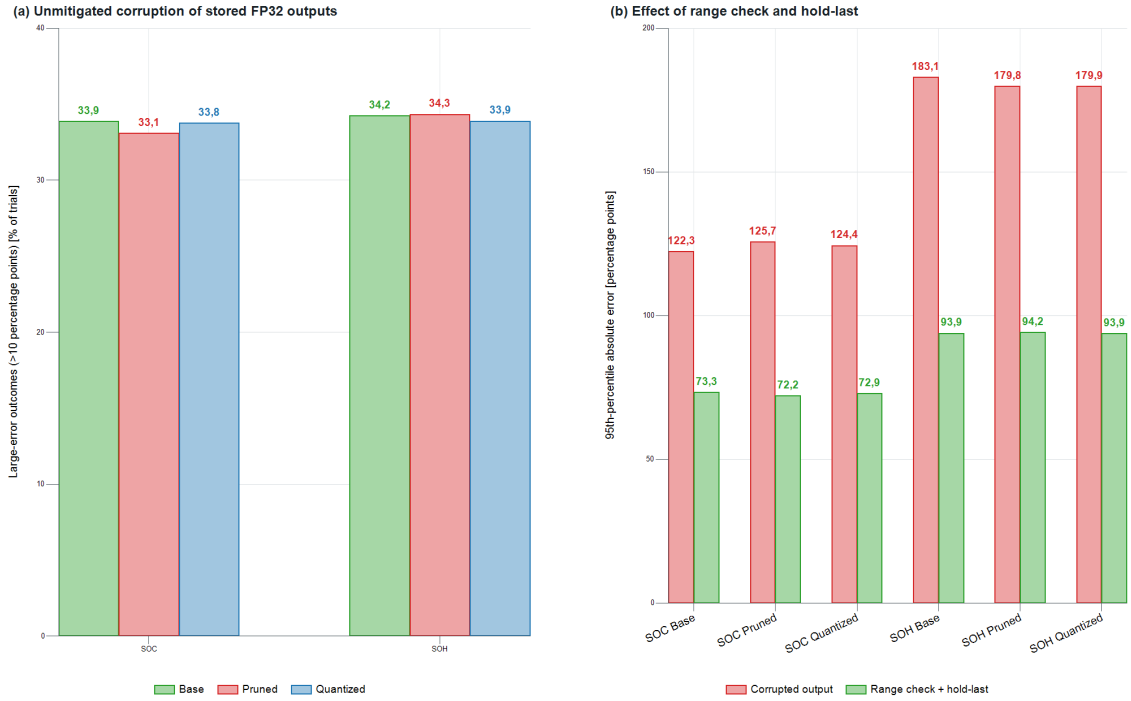


Figure A.20: Software-level sensitivity to random single-bit corruption of a saved FP32 estimator output, based on 20,000 trials per task and model. (a) Fraction of arbitrary-bit corruptions producing an absolute error above 10 percentage points. (b) P95 absolute error before and after rejecting non-finite or out-of-range values and holding the preceding prediction. The guard reduces extreme exponent and sign faults but cannot detect plausible in-range corruption. This experiment does not inject faults into MCU memory, recurrent states, weights, activations, inputs, or communication and is not an embedded fault-validation claim.

Appendix A.6. Analytical model-complexity scaling

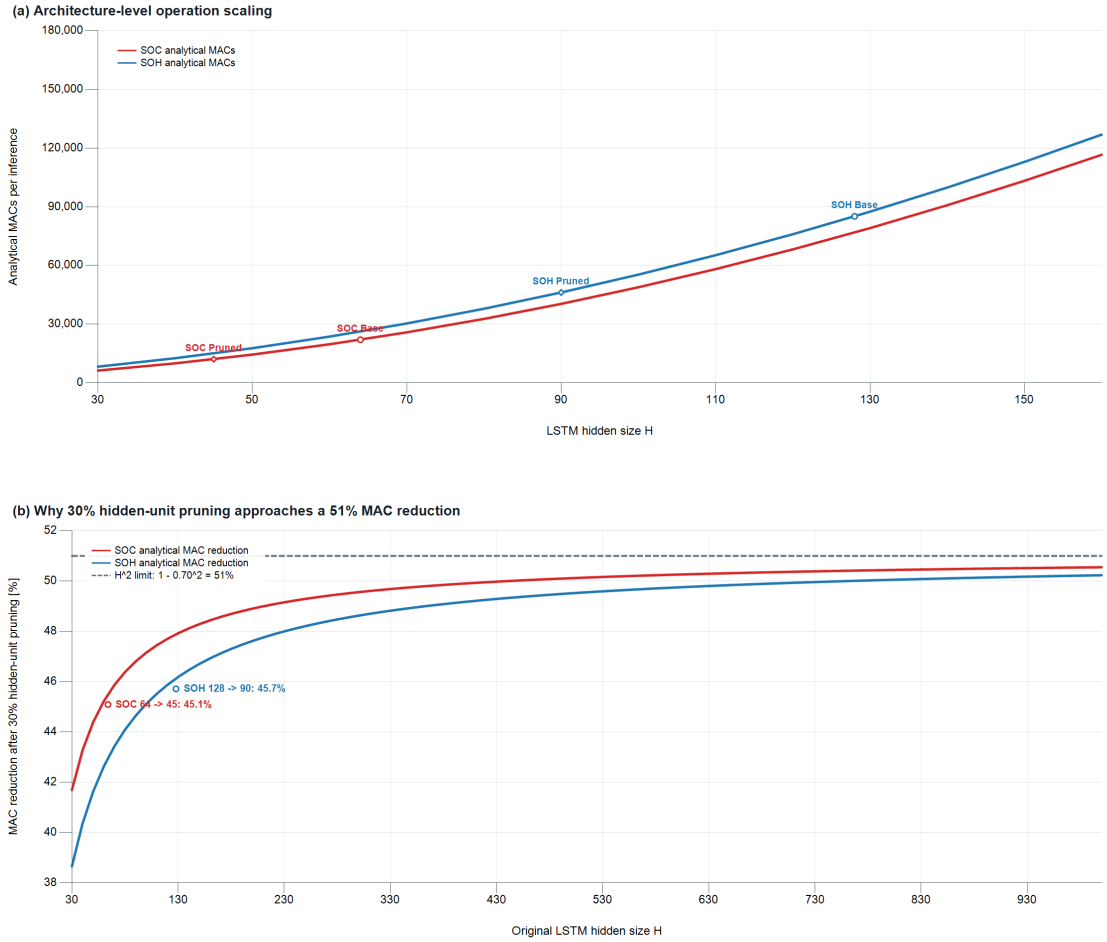
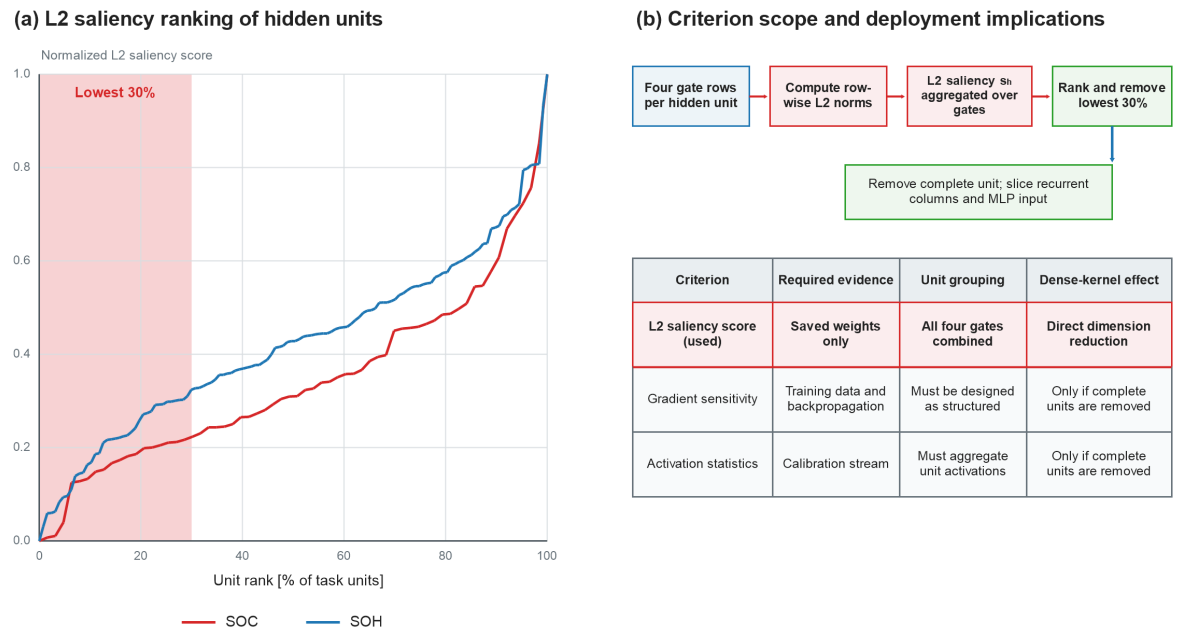


Figure A.21: Architecture-level operation scaling from Eq. **??**. (a) Analytical MACs per streaming inference over LSTM hidden size, with markers for the implemented SOC and SOH Base and Pruned dimensions. (b) MAC reduction for a nominal 30 % hidden-size reduction, $H' = 0.70H$. As the quadratic recurrent term dominates, the reduction approaches $1 - 0.70^2 = 51$ %; linear input/MLP and fixed output terms keep finite models below this limit. The figure reports analytical model operations, not measured flash, latency, energy, or cross-controller performance.

Appendix A.7. Scope of the L2 pruning criterion



Criterion	Required evidence	Unit grouping	Dense-kernel effect
L2 saliency score (used)	**Saved weights only**	**All four gates combined**	**Direct dimension reduction**
Gradient sensitivity	Training data and backpropagation	Must be designed as structured	Only if complete units are removed
Activation statistics	Calibration stream	Must aggregate unit activations	Only if complete units are removed

Figure A.22: Scope of the structured pruning criterion. (a) Normalised L2 saliency ranks for the SOC and SOH Base hidden units; the lowest 30 % define the task-specific removal sets. (b) The implemented path aggregates row-wise L2 norms across all four gates and removes complete recurrent units together with associated recurrent columns and MLP inputs. Gradient- and activation-based alternatives require additional evidence and would still need structured grouping to reduce a dense kernel. The comparison is methodological; no experimental ranking comparison was performed.

Appendix A.8. Verified mixed-precision boundary

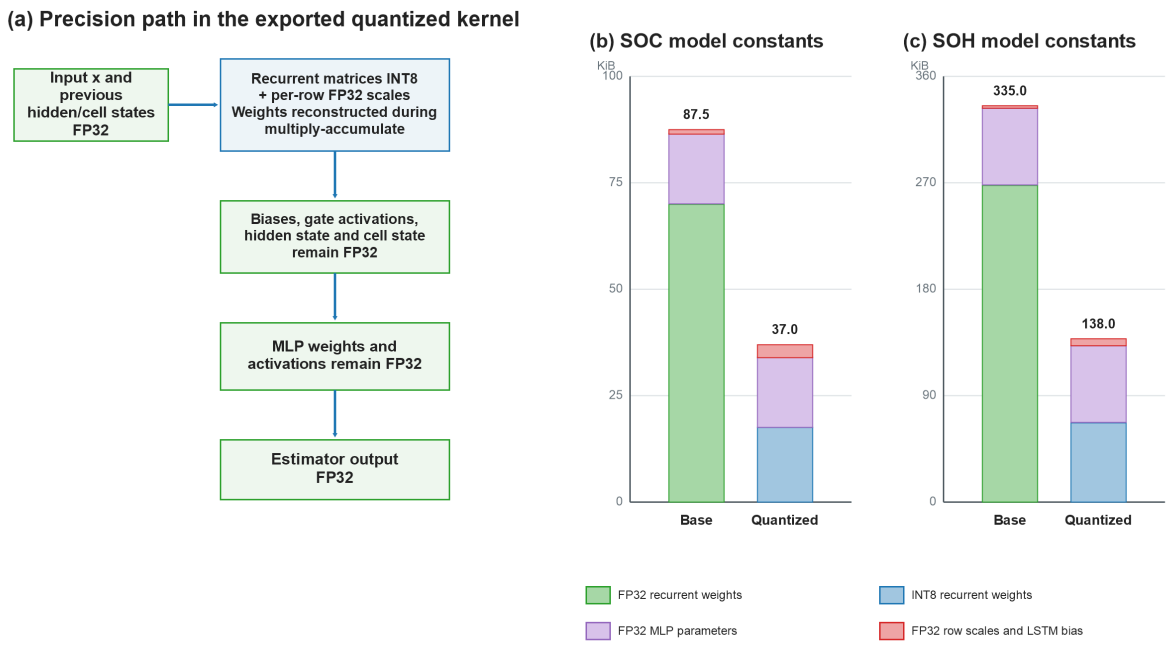


Figure A.23: Verified precision path and raw model-constant composition. (a) Only recurrent input-to-hidden and hidden-to-hidden matrices are stored as INT8; per-row scales, biases, gate activations, hidden/cell states, the MLP, and the estimator output remain FP32. (b,c) Analytical model constants for SOC and SOH Base and Quantized variants. Hidden/cell states and transient activations are runtime RAM and are not included in these bars. The totals describe model constants rather than complete linked-firmware flash; activation quantization and a fully integer kernel were not evaluated.

Appendix A.9. Static quantized-runtime accounting

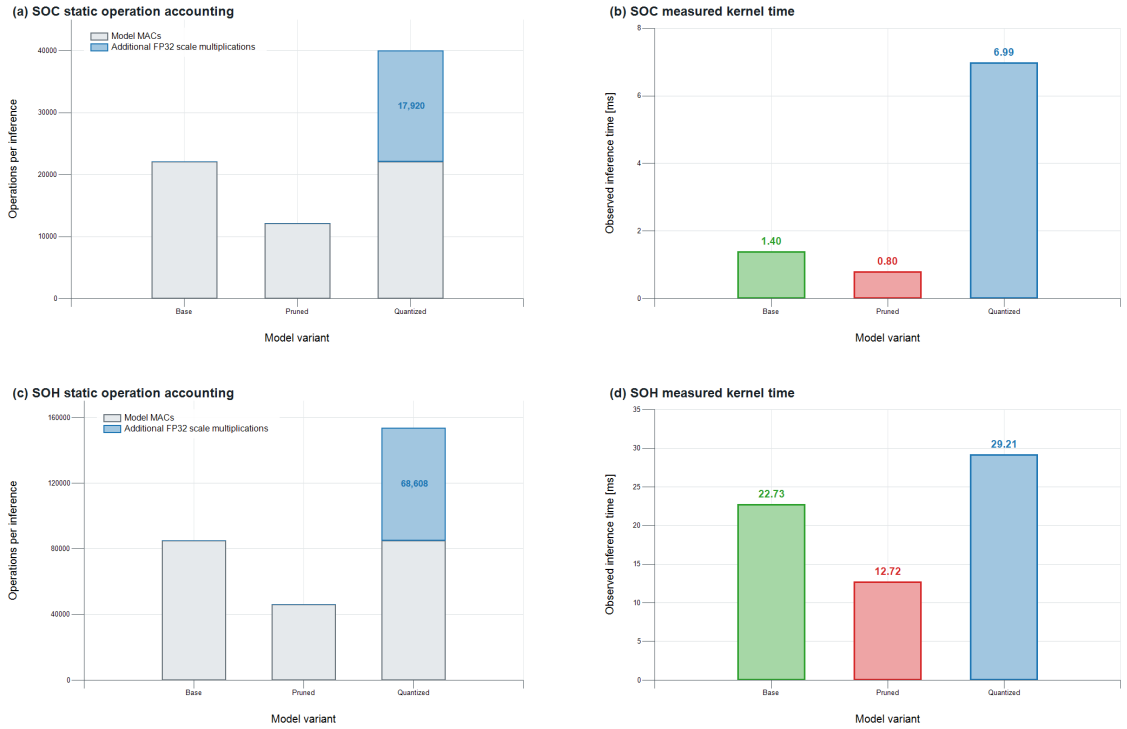


Figure A.24: Static source-level operation accounting beside measured STM32H753ZI kernel time. Panels (a,c) show model MACs and the additional FP32 row-scale multiplications implied by the current per-weight C expressions. Panels (b,d) show DWT measurements averaged over 10,000 streaming inferences. Static counts do not include INT8-to-FP32 conversion, indexing, loop, load/store, nonlinear-function, compiler, or memory-system costs and therefore are not a cycle-level or memory-bandwidth profile.

Funding

This research was funded by the German Federal Ministry for Education and Research (BMBF), grant number 03SF0684A (MG FARM). The authors are responsible for the content. The work was further supported by the German Research Foundation and the Open Access Publication Fund of the Technical University of Berlin.

Acknowledgements

We acknowledge the High Performance Computing Service of the Technical University of Berlin for providing the computational resources used in the pruning and quantization experiments. The authors thank Nils Strassenburg (Hasso-Plattner-Institut, Potsdam) for helpful discussions on pruning and quantization, Mano Schmitz for the collaboration on the dataset creation, and Max Mönikes and Pavel Aleinikau (Technical University of Berlin) for their collaboration on the BMS hardware.

CRediT authorship contribution statement

Florian Rzepka: Conceptualization, methodology, software, validation, formal analysis, investigation, data curation, writing–original draft preparation, writing–review and editing, visualization.

Julia Kowal: Resources, supervision, project administration, funding acquisition, writing–review and editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the author(s) used ChatGPT, DeepL, and Grammarly in order to improve the linguistic quality and style of the manuscript. After using these tools, the author(s) reviewed and edited the content as needed and take(s) full responsibility for the content of the published article.

Data availability

The LFP DoE Cycle Ageing Dataset used in this work can be obtained from the DOI repository in [**?**]. The landing page provides licensing information and download instructions.